Ramaiah Institute of Technology
Department of AI & DS

Data Communication & Networking

Unit 4

Unit 4

Network layer: Delivery, Forwarding, & Routing – Forwarding Techniques, Forwarding Process, Routing Table; Unicast routing protocols – Optimization, Intra and Inter domain routing, distance vector routing, link state routing, path vector routing.

Transport Layer - Process-to-Process delivery, User Datagram Protocol, Transmission Control Protocol.

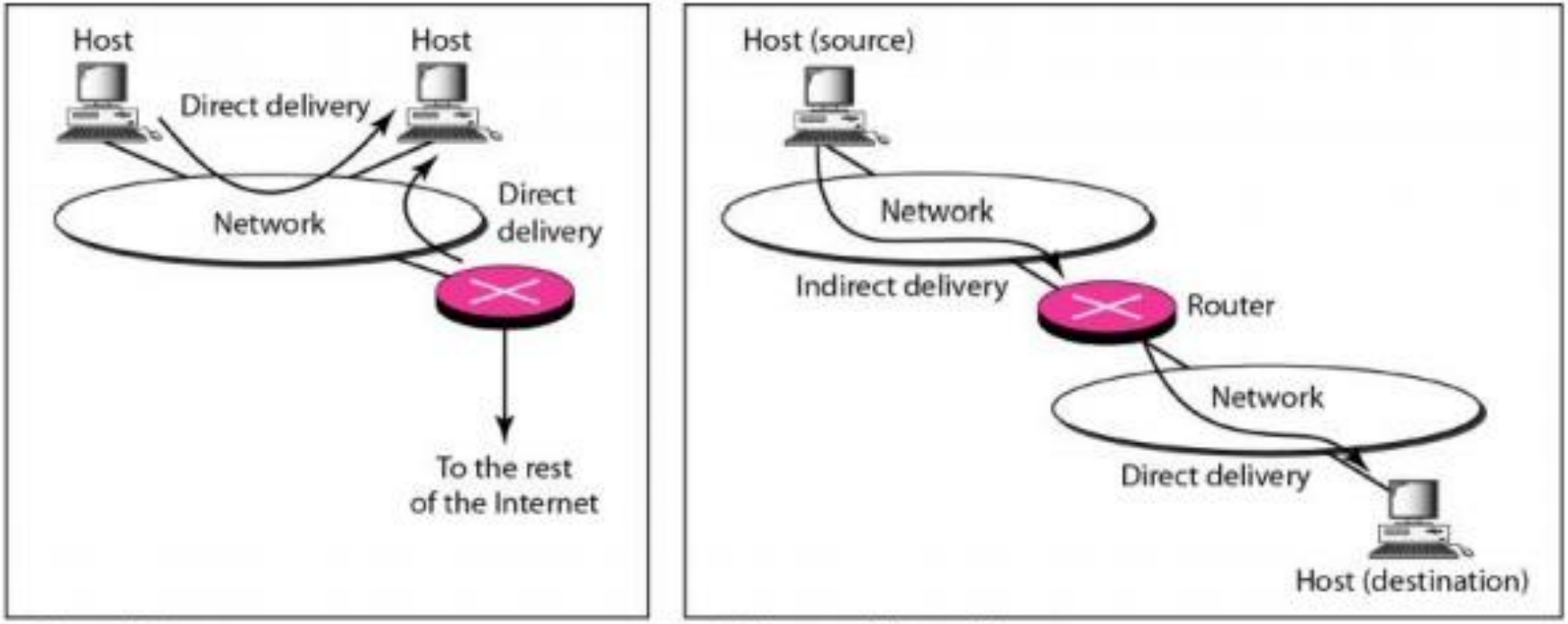
Delivery

- The network layer supervises the **handling of the packets** by the underlying physical networks.
- Two ways : Direct Versus Indirect Delivery
- Direct Packet Delivery:
- **Direct Delivery**
  - Direct delivery occurs when the source and destination of the packet are located on the same physical network or when the delivery is between the last router and the destination host.
  - The sender can extract the network address of the destination (using the mask) and compare this address with the addresses of the networks to which it is connected. If a match is found, the delivery is direct.

Delivery

- **Indirect Delivery**
- If the destination host is not on the same network as the deliverer, the packet is delivered indirectly.
- The packet goes from router to router until it reaches the one connected to the same physical network as its final destination.

Delivery



a. Direct delivery

b. Indirect and direct delivery

Figure 3.39 Direct and indirect delivery

FORWARDING

- Forwarding means to **place the packet in its route to its destination.**
- Forwarding requires a host or a router to have a **routing table.**
- When a host has a packet to send or when a router has received a packet to be forwarded, it looks at this table to **find the route to the final destination**.
- But the **number of entries** needed in the routing table would make table **lookups inefficient**.

Forwarding Techniques

- Several **techniques** can make the size of the routing table manageable and also handle issues such as security.

Next-Hop Method Versus Route Method

- In this technique, the routing table holds only the address of the next hop instead of information about the complete route (route method). The entries of a routing table must be consistent with one another.

Figure 22.2 *Route method versus next-hop method*

a. Routing tables based on route

Routing table for host A

Destination	Route
HostB	R1, R2, host B

Routing table for R1

Destination	Route
HostB	R2, host B

Routing table for R2

Destination	Route
HostB	HostB

b. Routing tables based on next hop

Routing table for host A

Destination	Next hop
Host B	R1

Routing table for R1

Destination	Next hop
HostB	R2

Routing table for R2

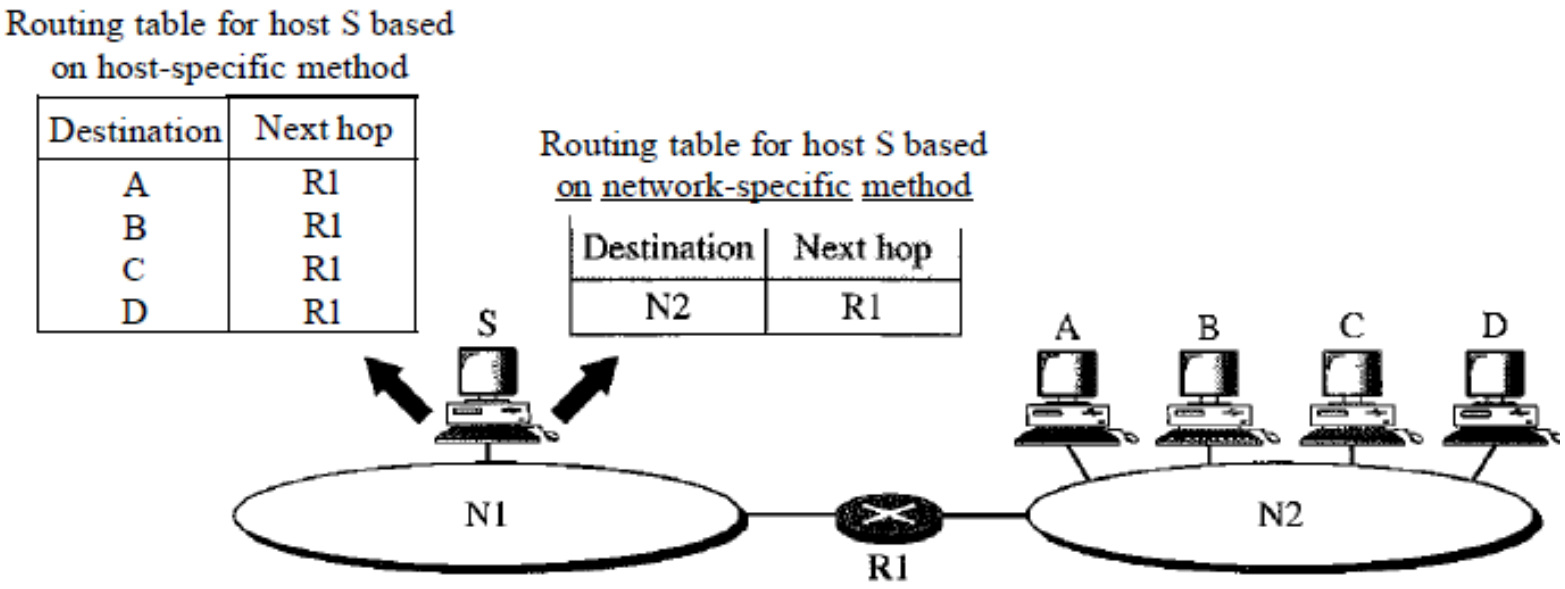
Destination	Next hop
Host B	

Forwarding Techniques

Network-Specific Method Versus Host-Specific Method

- In network-specific method, instead of having an entry for every destination host connected to the same physical network (host-specific method), there is only one entry that defines the address of the destination network itself.
- Treat all hosts connected to the same network as one single entity.

Figure 22.3 *Host-specific versus network-specific method*

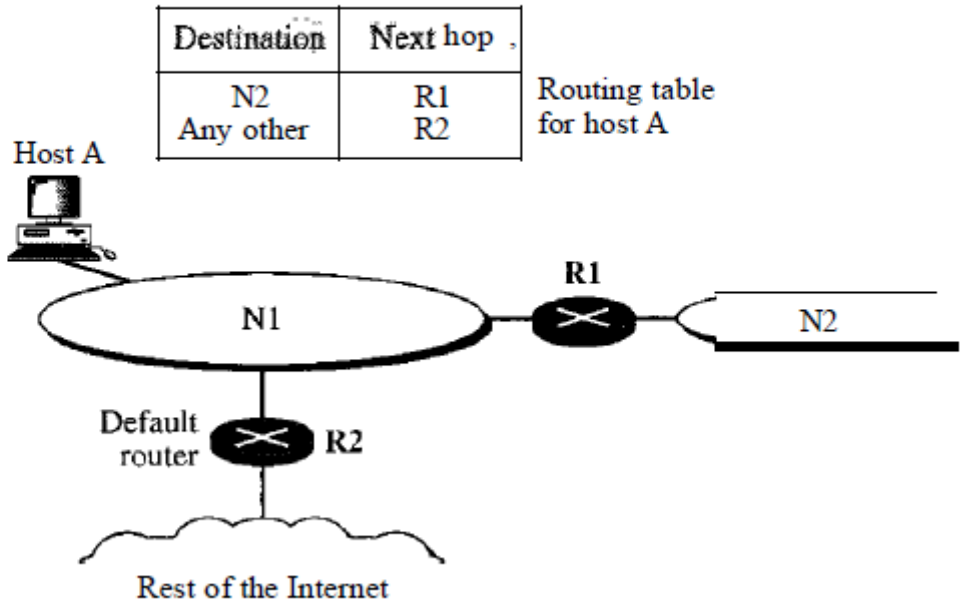


Forwarding Techniques

Default Method

- Another technique to simplify routing is called the default method.
- Host A is connected to a network with two routers. Router Rl routes the packets to hosts connected to network N2.
- For the rest of the Internet, router R2 is used.
- So instead of listing all networks in the entire Internet, host A can just have one entry called the default.

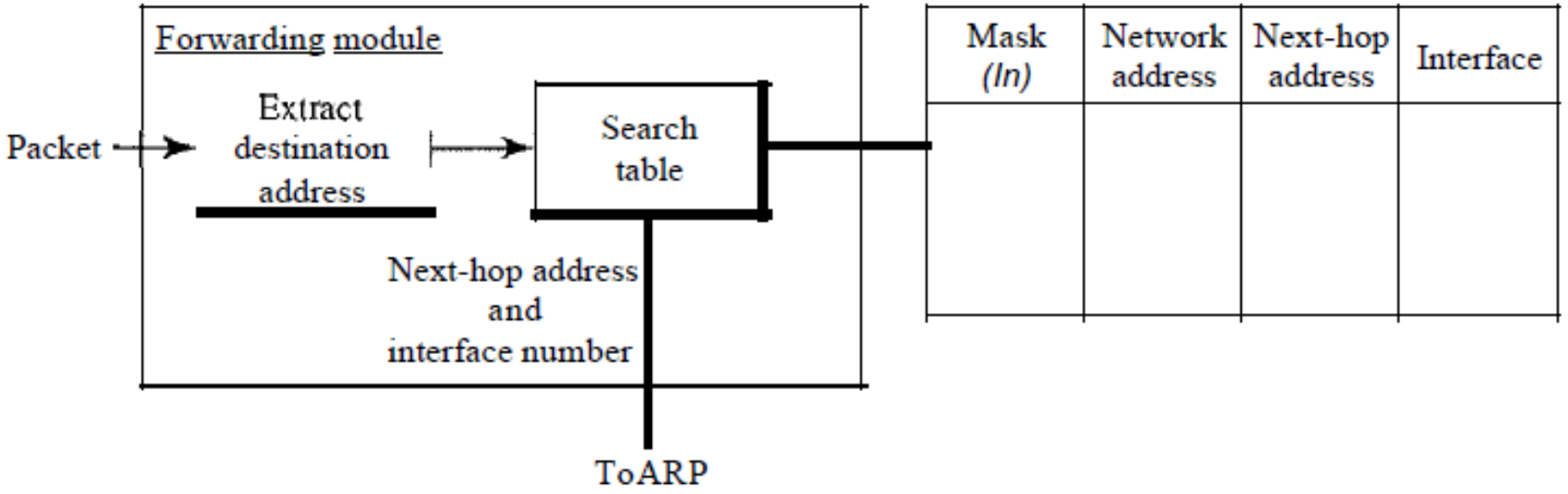
Figure 22.4 *Default method*



Forwarding Process

- Hosts and routers use classless addressing.
- The routing table needs to have **one row of information for each block involved**.
- The table is searched based on the **network address** (first address in the block).
- Destination address in the packet gives no clue about the network address.
- So, include the **mask (/n)** in the table

Figure 22.5 *Simplified forwarding module in classless address*



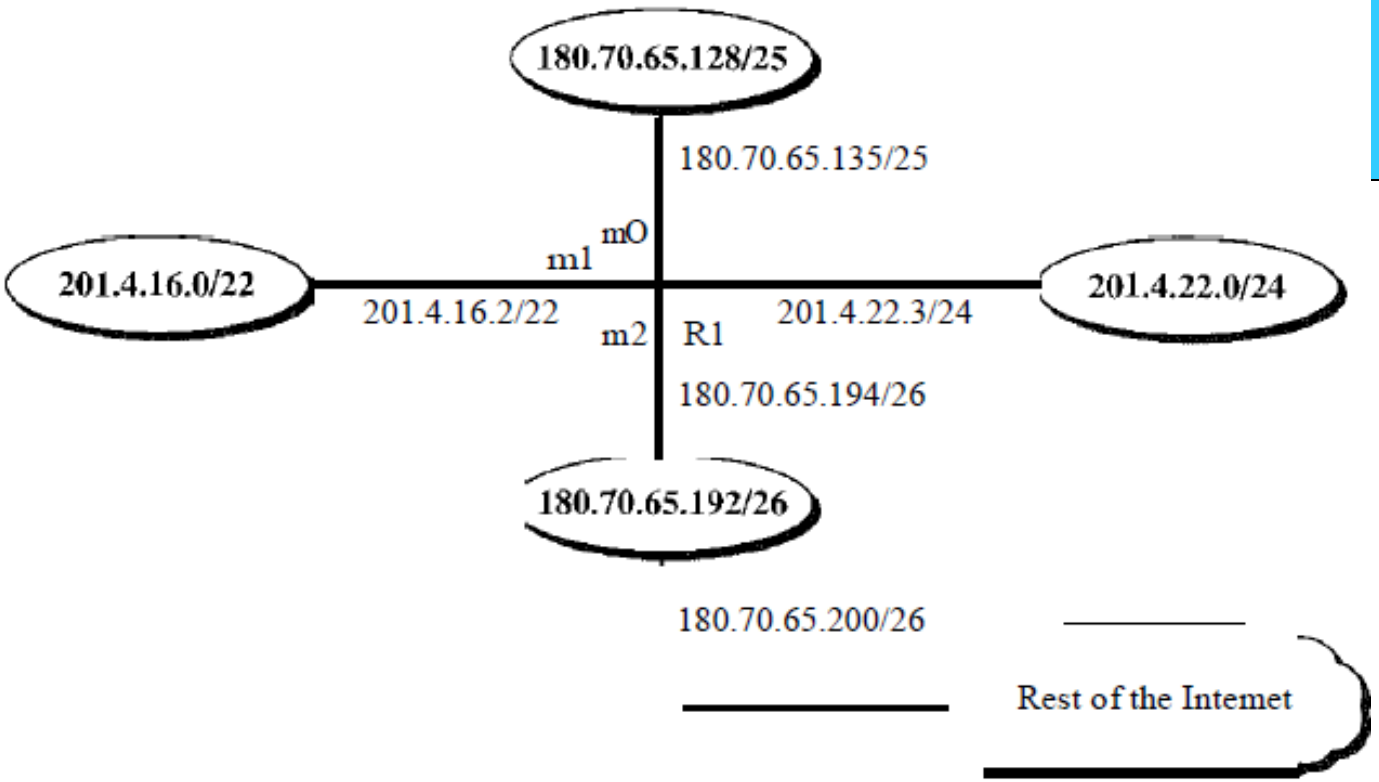


Table 22.1 *Routing table for router RI in Figure 22.6*

Mask	*Network Address*	*Next Hop*	*Interface*
/26	180.70.65.192	-	m2
/25	180.70.65.128	-	mO
/24	201.4.22.0	-	m3
/22	201.4.16.0	....	m1
Any	Any	180.70.65.200	m2

Forwarding Process

Show the forwarding process if a packet arrives at R1 in Figure 22.6 with the destination address 180.70.65.140.

Solution

The router performs the following steps:

1. The first mask (/26) is applied to the destination address. The result is 180.70.65.128, which does not match the corresponding network address.
2. The second mask (/25) is applied to the destination address. The result is 180.70.65.128, which matches the corresponding network address. The next-hop address (the destination address of the packet in this case) and the interface number mO are passed to ARP for further processing.

Example 22.3

Show the forwarding process if a packet arrives at R1 in Figure 22.6 with the destination address 201.4.22.35.

Solution

The router performs the following steps:

1. The first mask (/26) is applied to the destination address. The result is 201.4.22.0, which does not match the corresponding network address (row 1).
2. The second mask (/25) is applied to the destination address. The result is 201.4.22.0, which does not match the corresponding network address (row 2).
3. The third mask (/24) is applied to the destination address. The result is 201.4.22.0, which matches the corresponding network address. The destination address of the packet and the interface number m3 are passed to ARP.

Table 22.1 *Routing table for router RI in Figure 22.6*

Mask	*Network Address*	*Next Hop*	*Interface*
/26	180.70.65.192	-	m2
/25	180.70.65.128	-	mO
/24	201.4.22.0	-	m3
/22	201.4.16.0	....	m1
Any	Any	180.70.65.200	m2

Forwarding Process

Example 22.4

Show the forwarding process if a packet arrives at R1 in Figure 22.6 with the destination address 18.24.32.78.

Solution

This time all masks are applied, one by one, to the destination address, but no matching network address is found. When it reaches the end of the table, the module gives the next-hop address 180.70.65.200 and interface number m2 to ARP. This is probably an outgoing package that needs to be sent, via the default router, to someplace else in the Internet.

Table 22.1 *Routing table for router RI in Figure 22.6*

Mask	*Network Address*	*Next Hop*	*Interface*
/26	180.70.65.192	-	m2
/25	180.70.65.128	-	mO
/24	201.4.22.0	-	m3
/22	201.4.16.0	....	m1
Any	Any	180.70.65.200	m2

Routing Table

The routing table can be either static or dynamic.

Static Routing Table

- A static routing table contains information entered manually.
- The **administrator** enters the route for each destination into the table. When a table is created, it **cannot update automatically** when there is a change in the Internet. The table must be **manually altered** by the administrator.
- A static routing table can be used in a **small internet** that does not change very often, or in an **experimental internet** for **troubleshooting**.
- It is **poor strategy** to use a static routing table in a big internet such as the Internet.

Routing Table

Dynamic Routing Table

- A dynamic routing table is updated periodically by using one of the dynamic routing protocols such as RIP, OSPF, or BGP.
- Whenever there is a change in the Internet, such as a shutdown of a router or breaking of a link, the dynamic routing protocols update all the tables in the routers & host automatically.
- The routers in the Internet need to be updated dynamically for efficient delivery of the IP packets.

Routing Table

Format

- A routing table for classless addressing has a minimum of four columns. However, some of today's routers have even more columns.
- Number of columns is vendor-dependent, and not all columns can be found in all routers.

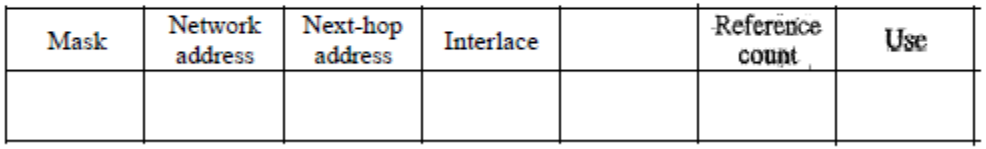
Figure 22.10 *Common fields in a routing table*

Mask	Network address	Next-hop address	Interlace		Reference count	Use

Mask. This field defines the mask applied for the entry.
Network address. This field defines the network address to which the packet is finally delivered. In the case of host-specific routing, this field defines the address of the destination host.
Next-hop address. This field defines the address of the next-hop router to which the packet is delivered.
Interface. This field shows the name of the interface.

Routing Table

Figure 22.10 *Common fields in a routing table*



- **Flags**. This field defines up to five flags. Flags are on/off switches that signify either presence or absence. The five flags are U (up), G (gateway), H (host-specific), D (added by redirection), and M (modified by redirection).

a. **U (up).** The U flag indicates the router is up and running. If this flag is not present, it means that the router is down. The packet cannot be forwarded and is discarded.

b. **G (gateway).** The G flag means that the destination is in another network. The packet is delivered to the next-hop router for delivery (indirect delivery). When this flag is **missing**, it means the destination is in this network (direct delivery).

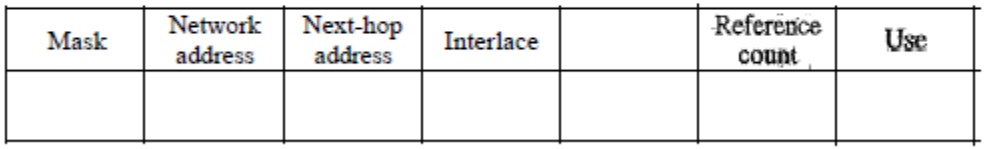
c. **H (host-specific**). The H flag indicates that the entry in the network address field is a host-specific address. When it is missing, it means that the address is only the network address of the destination.

d. **D (added by redirection**). The D flag indicates that routing information for this destination has been added to the host routing table by a **redirection message from ICMP**.

e. **M (modified by redirection**). The M flag indicates that the routing information for this destination has been modified by a **redirection message from ICMP**.

Routing Table

Figure 22.10 *Common fields in a routing table*



- **Reference count:** This field gives the number of users of this route at the moment. For example, if five people at the same time are connecting to the same host from this router, the value of this column is 5.
- **Use:** This field shows the number of packets transmitted through this router for the corresponding destination.

Unicast Routing Protocols

- A **routing protocol** is a combination of rules and procedures that lets routers in the internet inform each other of changes.
- It allows routers to **share whatever** they know about the internet or their neighborhood.
- The routing protocols also include **procedures for combining information** received from other routers.

Optimization

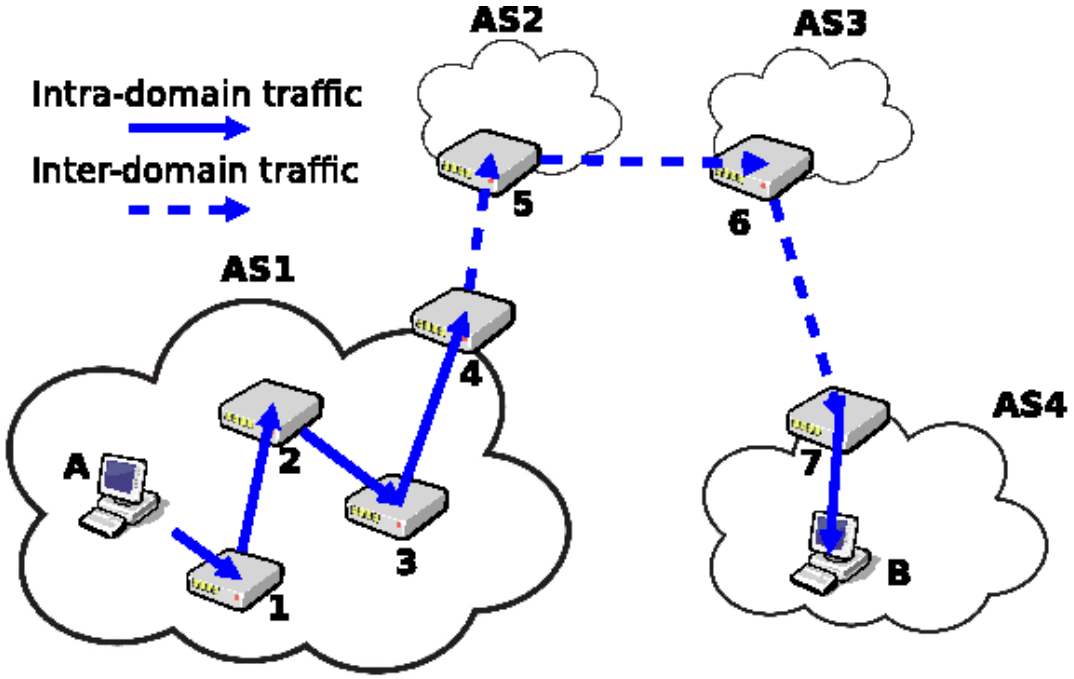
- A router receives a packet from a network and passes it to another network. A router is usually attached to several networks. When it receives a packet, to which network should it pass the packet?
- The decision is based on **optimization. The approaches are:**
- To assign a cost for passing through a network. Call this cost a metric.
- The metric assigned to each network depends on the type of protocol.
- Some simple protocols, such as the **Routing Information Protocol (RIP),** treat all networks as equals. The cost of passing through a network is the same; it is one **hop count**. So if a packet passes through 10 networks to reach the destination, the total cost is 10 hop counts.

Unicast Routing Protocols

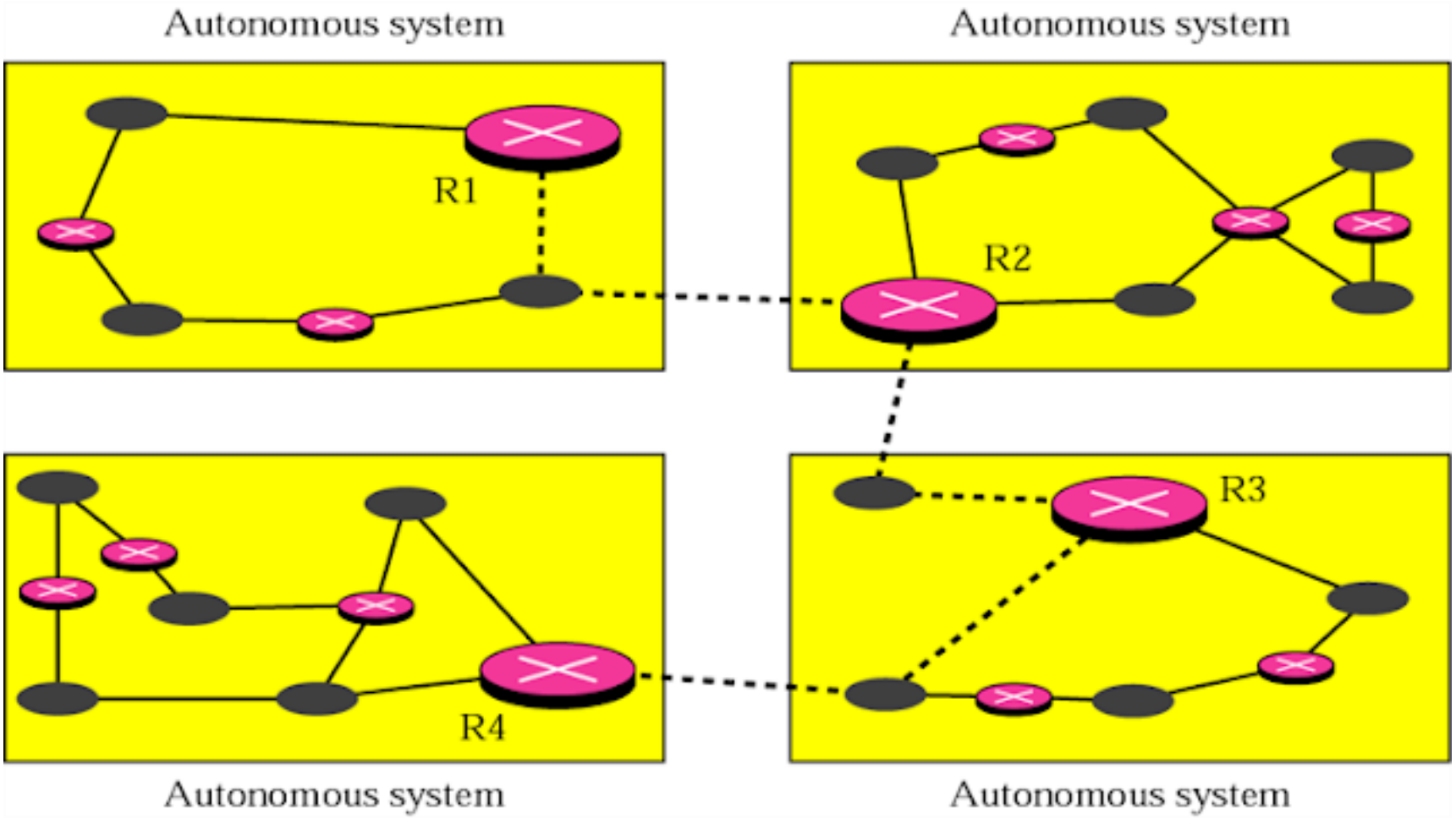
- **Open Shortest Path First (OSPF),** allow the administrator to assign a cost for passing through a network based on **the type of service** required.
  - A route through a network can have different costs (metrics).
  - For example, if **maximum throughput** is the desired type of service, a **satellite link has a lower metric than a fiber-optic line.**
  - If **minimum delay** is the desired type of service, a **fiber-optic line has a lower metric than a satellite link.**
  - OSPF protocol allows each router to have several routing tables based on the required type of service.
- The **Border Gateway Protocol (BGP),** the criterion is the policy, which can be set by the administrator. The policy defines what paths should be chosen.

Intra- and Interdomain Routing

- Today, an internet can be so large that one routing protocol cannot handle the task of updating the routing tables of all routers.
- So, an internet is divided into **autonomous systems**. An autonomous system (AS) is a **group of networks** and routers under the authority of a single administration.
- **Routing inside** an autonomous system is referred to as Intradomain routing. **Routing between** autonomous systems is referred to as Interdomain routing.
- Each autonomous system can choose **one or more Intradomain routing protocols** to handle routing inside the autonomous system.



Intra- and Interdomain Routing



Intra- and Interdomain Routing

Interdomain and Intradomain Routing

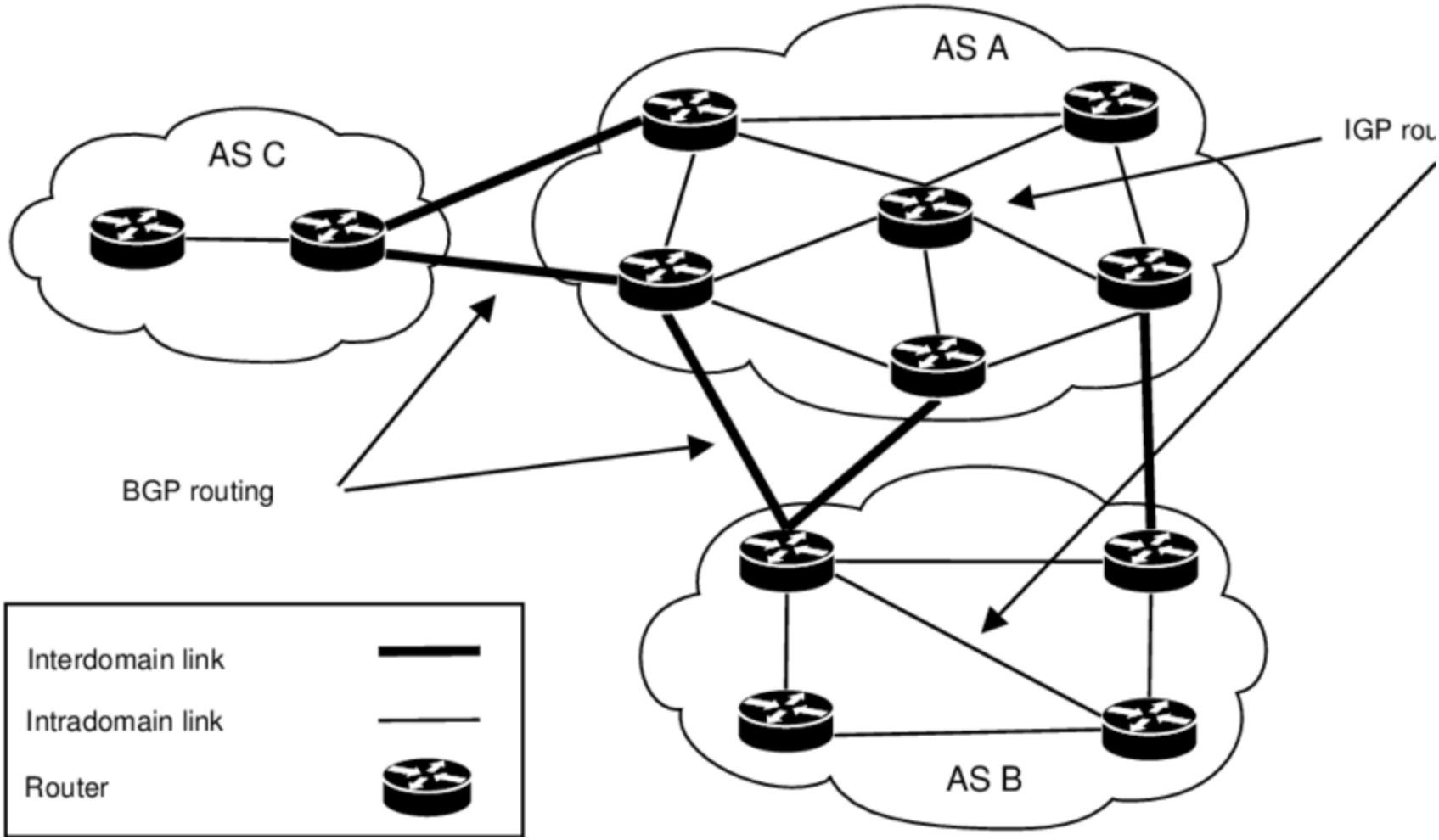
Intradomain Routing

- Routing within an AS
- Ignores the Internet outside the autonomous system
- Protocols for Intradomain routing are also called **Interior Gateway Protocols** or **IGP's**.
- Popular protocols are
  - RIP (simple, old)
  - OSPF (better)

Interdomain Routing

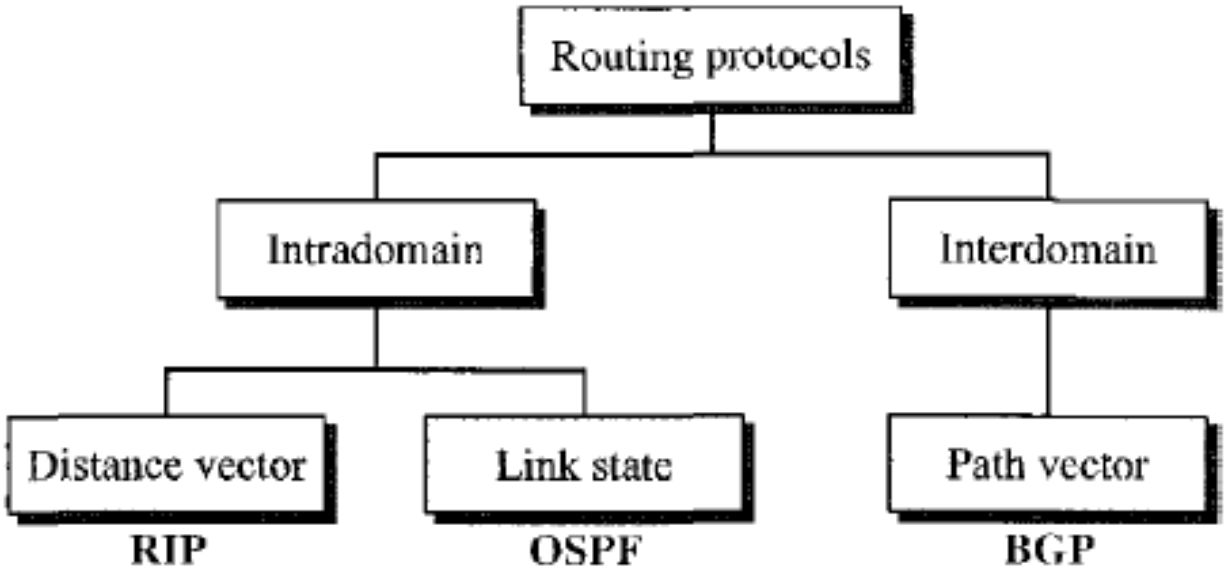
- Routing between AS's
- Assumes that the Internet consists of a collection of interconnected AS's
- Protocols for interdomain routing are also called **Exterior Gateway Protocols** or **EGP's.**
- Routing protocol:
  - BGP

Intra- and Interdomain Routing



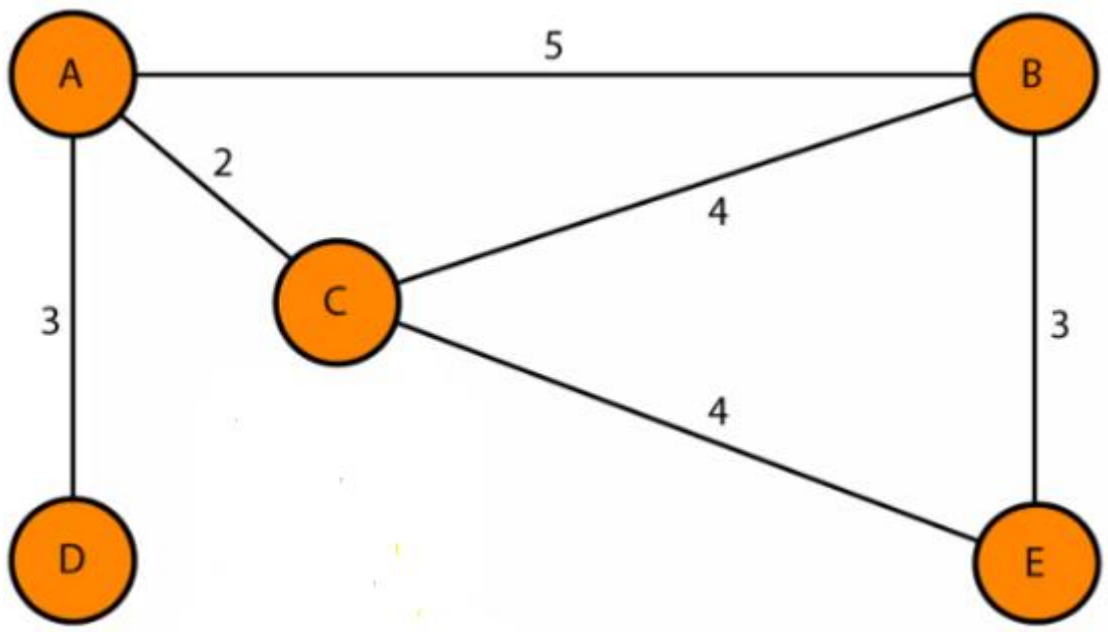
Intra- and Interdomain Routing

Figure 22.13 *Popular routing protocols*



Distance Vector Routing

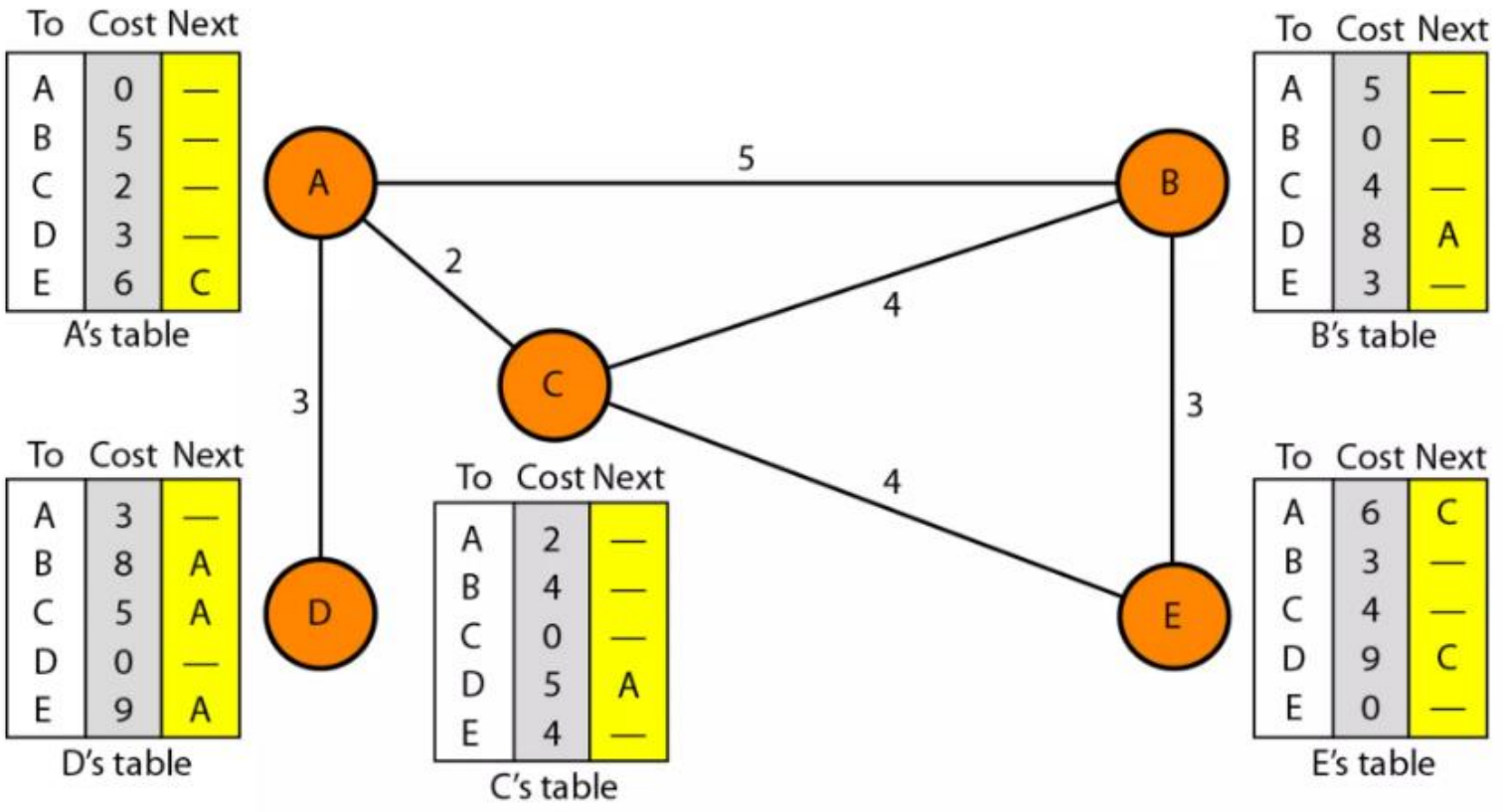
- In distance vector routing, the least-cost route between any two nodes is the route with minimum distance.



Distance Vector Routing

- Each node maintains a vector (table) of minimum distances to every node. The table at each node also guides the packets to the desired node by showing the next stop in the route (next-hop routing).

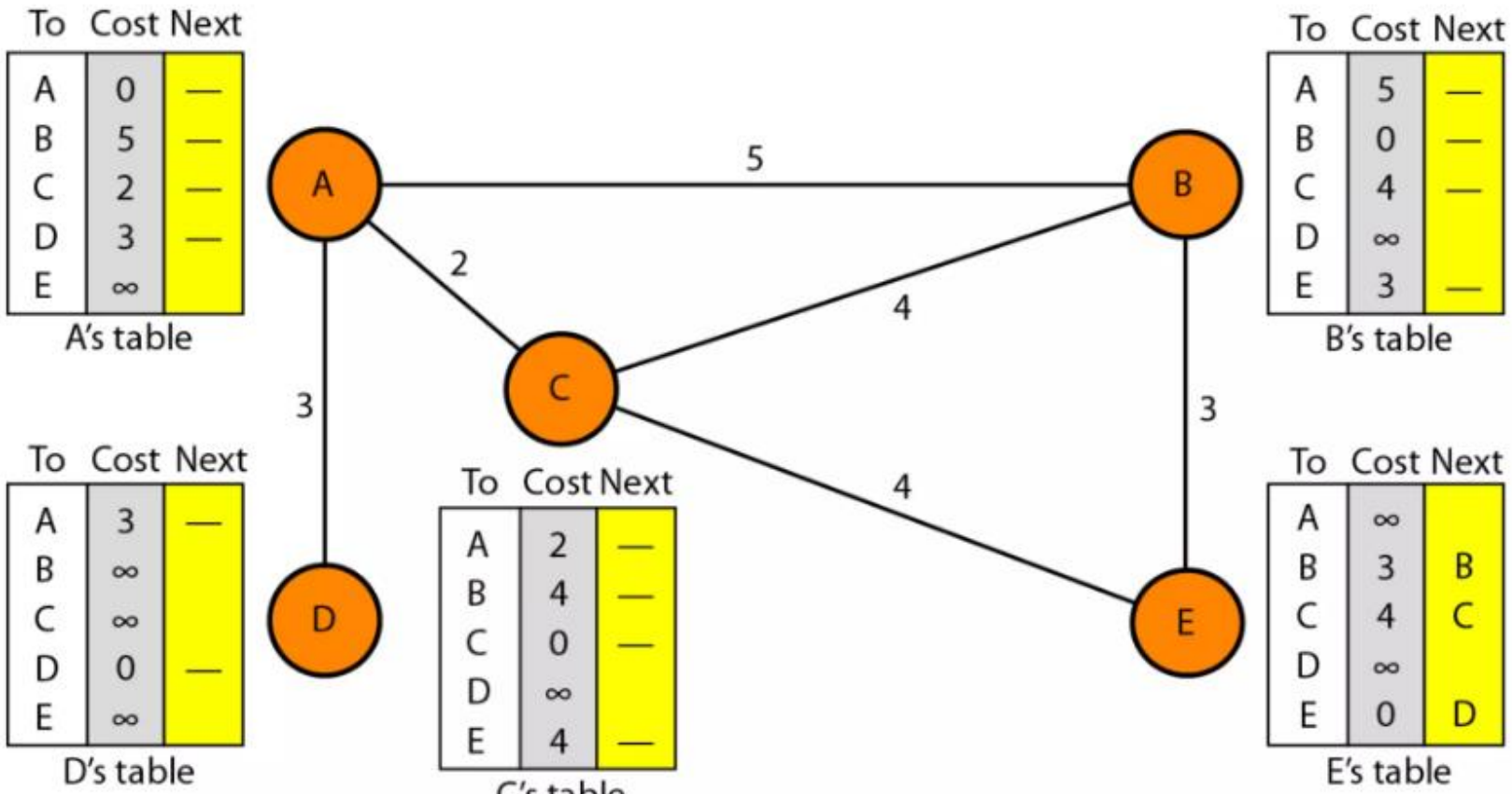
Figure 22.14 *Distance vector routing tables*



Distance Vector Routing

Initialization: Each node knows how to reach any other node and the cost. At the beginning, Each node can know only the distance between itself and its immediate neighbors, those directly connected to it.

Figure 22.15 *Initialization of tables in distance vector routing*



Distance Vector Routing

- **Sharing**
- The whole idea of distance vector routing is the sharing of information between neighbors.
- Although node A does not know about node E, node C does. So if node C shares its routing table with A, node A can also know how to reach node E.
- Node C does not know how to reach node D, but node A does. If node A shares its routing table with node C, node C also knows how to reach node D.
- There is only one problem. How much of the table must be shared with each neighbor?
- A node is not aware of a neighbor's table. The best solution for each node is to send its entire table to the neighbor and let the neighbor decide what part to use and what part to discard.

Distance Vector Routing

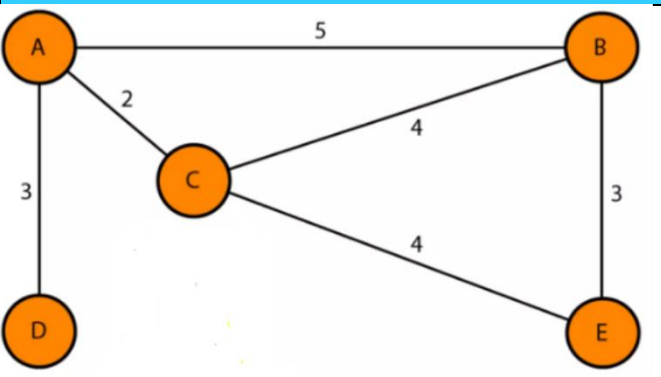
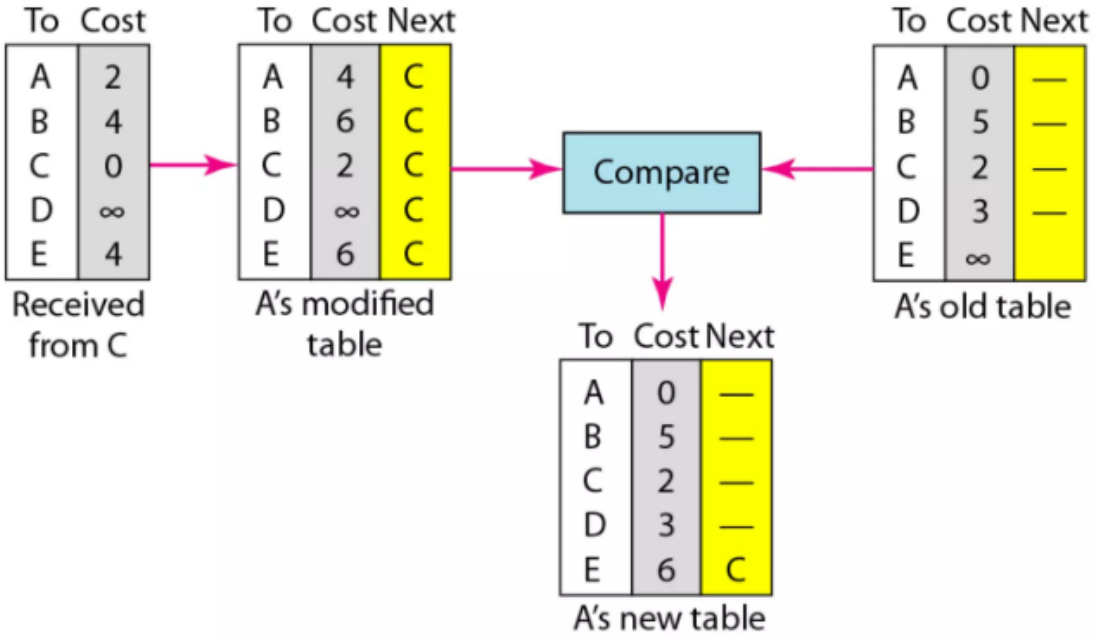


Figure 22.16 *Updating in distance vector routing*



Distance Vector Routing

- **Updating**
- When a node receives a two-column table from a neighbor, it needs to update its routing table. Updating takes three steps:

1. The receiving node needs to add the cost between itself and the sending node to each value in the second column. The logic is clear. If node C claims that its distance to a destination is x mi, and the distance between A and C is y mi, then the distance between A and that destination, via C, is x + y mi.
2. The receiving node needs to add the name of the sending node to each row as the third column if the receiving node uses information from any row. The sending node is the next node in the route.
3. The receiving node needs to **compare each row** of its old table with the corresponding row of the modified version of the received table.

    a. If the **next-node entry is different**, the receiving node chooses the row with the smaller cost. If there is a tie, the old one is kept.

    b. If the **next-node entry is the same**, the receiving node chooses the new row. For example, suppose node C has previously advertised a route to node X with distance

Distance Vector Routing

- **There are several points we need to emphasize here.**

1. When we add any number to infinity, the result is still infinity.
2. The modified table shows how to reach A from A via C. If A needs to reach itself via C, it needs to go to C and come back, a distance of 4.
3. The only benefit from this updating of node A is the last entry, how to reach E. Previously, node A did not know how to reach E (distance of infinity); now it knows that the cost is 6 via C.

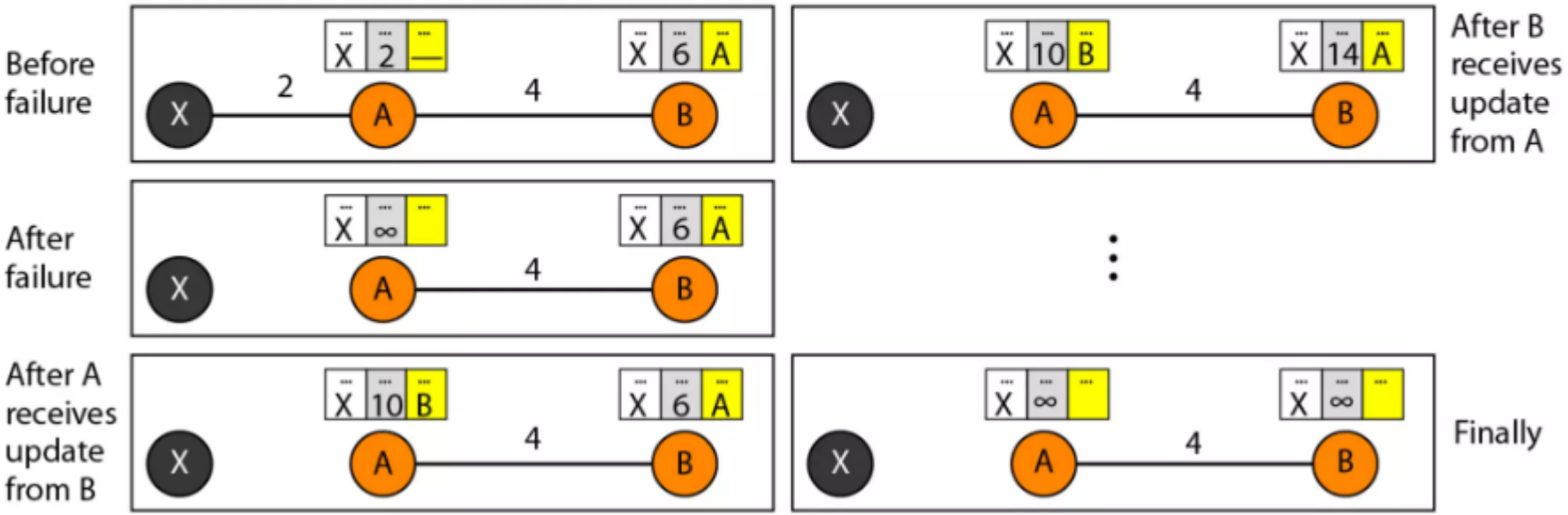
Distance Vector Routing

When to Share

- The question now is, When does a node send its partial routing table (only two columns) to all its immediate neighbors? The table is sent both periodically and when there is a change in the table.
- **Periodic Update** A node sends its routing table, normally every 30 s, in a periodic update. The period depends on the protocol that is using distance vector routing.
- **Triggered Update** A node sends its two-column routing table to its neighbors anytime there is a change in its routing table. This is called a triggered update. The change can result from the following.
  - A node receives a table from a neighbor, resulting in changes in its own table after updating.
  - A node detects some failure in the neighboring links which results in a distance change to infinity.

Distance Vector Routing

Figure 22.17 *Two-node instability*



Distance Vector Routing: Drawbacks

Two-Node Loop Instability

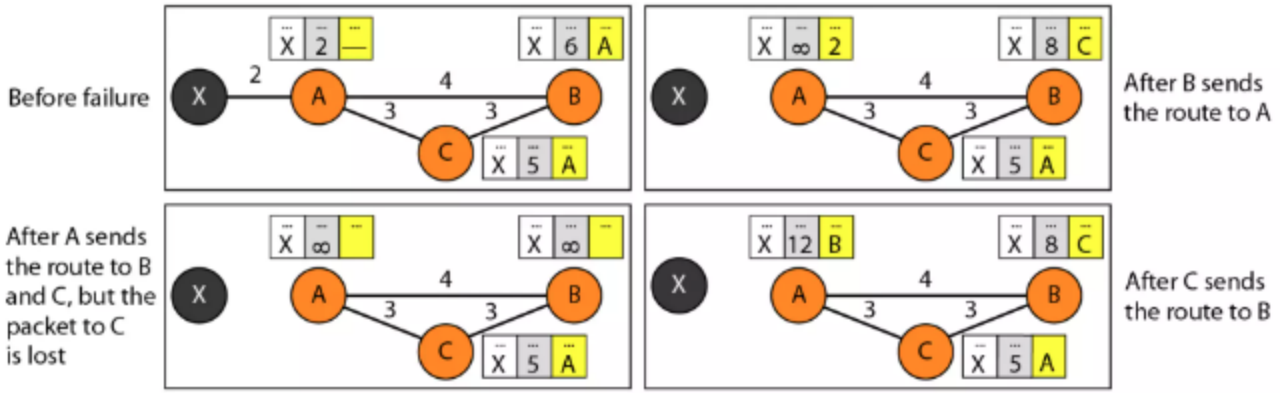
- A problem with distance vector routing is instability, which means that a network using this protocol can become unstable.
- At the beginning, both nodes A and B know how to reach node X. But suddenly, the link between A and X fails. Node A changes its table.
- If A can send its table to B immediately, everything is fine. However, the system becomes unstable if B sends its routing table to A before receiving A's routing table.
- Node A receives the update and, assuming that B has found a way to reach X, immediately updates its routing table. Based on the triggered update strategy, A sends its new update to B.
- Now B thinks that something has been changed around A and updates its routing table. The cost of reaching X increases gradually until it reaches infinity. At this moment, both A and B know that X cannot be reached. However, during this time the system is not stable.
- Packets bounce between A and B, creating a two-node loop problem.
- A few solutions have been proposed for instability of this kind.

Distance Vector Routing: Drawbacks

Defining Infinity : The first obvious solution is to redefine infinity to a smaller number, such as 100. For our previous scenario, the system will be stable in less than 20 updates.

Split Horizon Another solution is called split horizon. In this strategy, instead of flooding the table through each interface, each node sends only part of its table through each interface. If, according to its table, node B thinks that the optimum route to reach X is via A, it does not need to advertise this piece of information to A; the information has come from A (A already knows). Taking information from node A, modifying it, and sending it back to node A creates the confusion.

Distance Vector Routing: Drawbacks



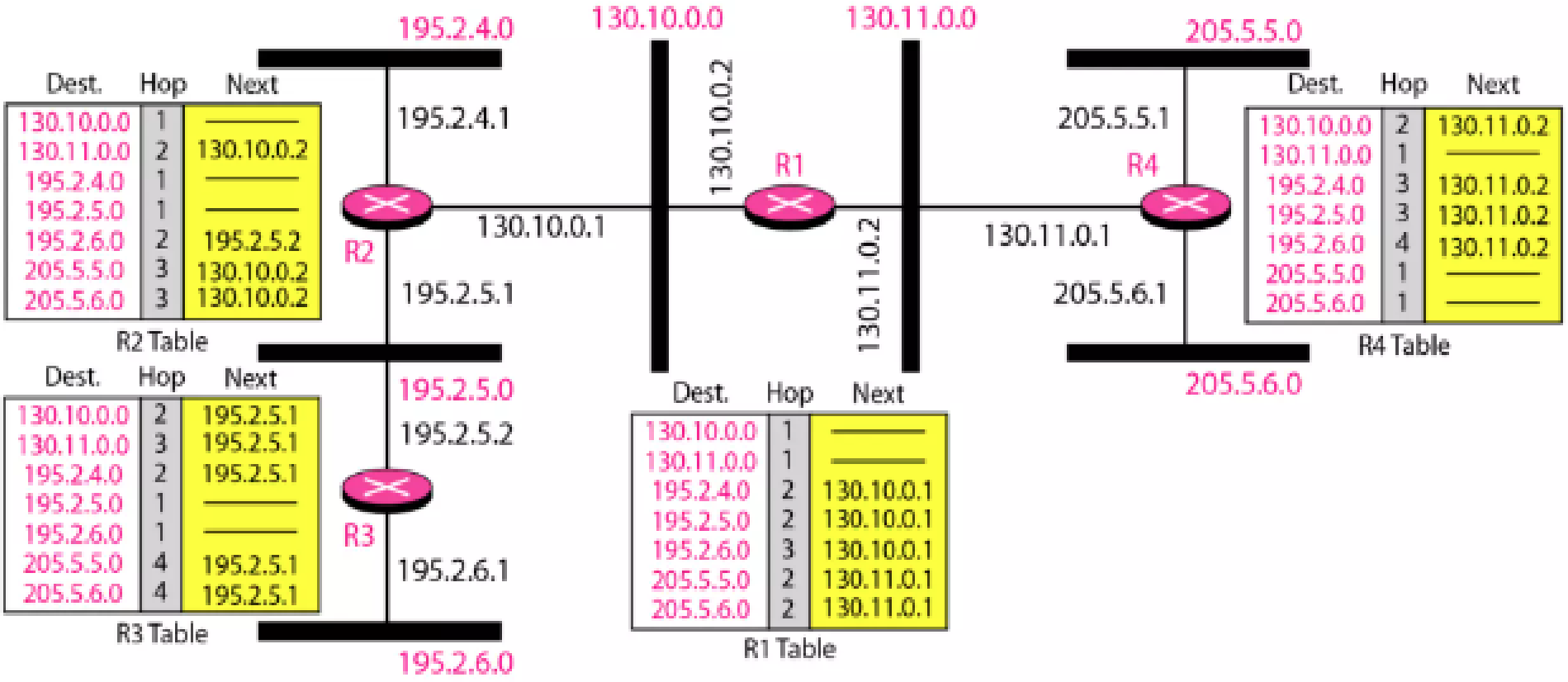
Routing Information Protocol

- The Routing Information Protocol (RIP) is an intradomain routing protocol used
- inside an autonomous system.
- It is a very simple protocol based on distance vector routing.
- RIP implements distance vector routing directly with some considerations:

1. In an autonomous system, we are dealing with routers and networks (links). The routers have routing tables; networks do not.
2. The destination in a routing table is a network, which means the first column defines a network address.
3. The metric used by RIP is very simple; the distance is defined as the number of links (networks) to reach the destination. For this reason, the metric in RIP is called a **hop count.**
4. **Infinity is defined as 16**, which means that any route in an autonomous system using RIP cannot have more than 15 hops.
5. The next-node column defines the **address of the router** to which the packet is to be sent to reach its destination.

Routing Information Protocol

Figure 22.19 *Example of a domain using RIP*



Routing Information Protocol

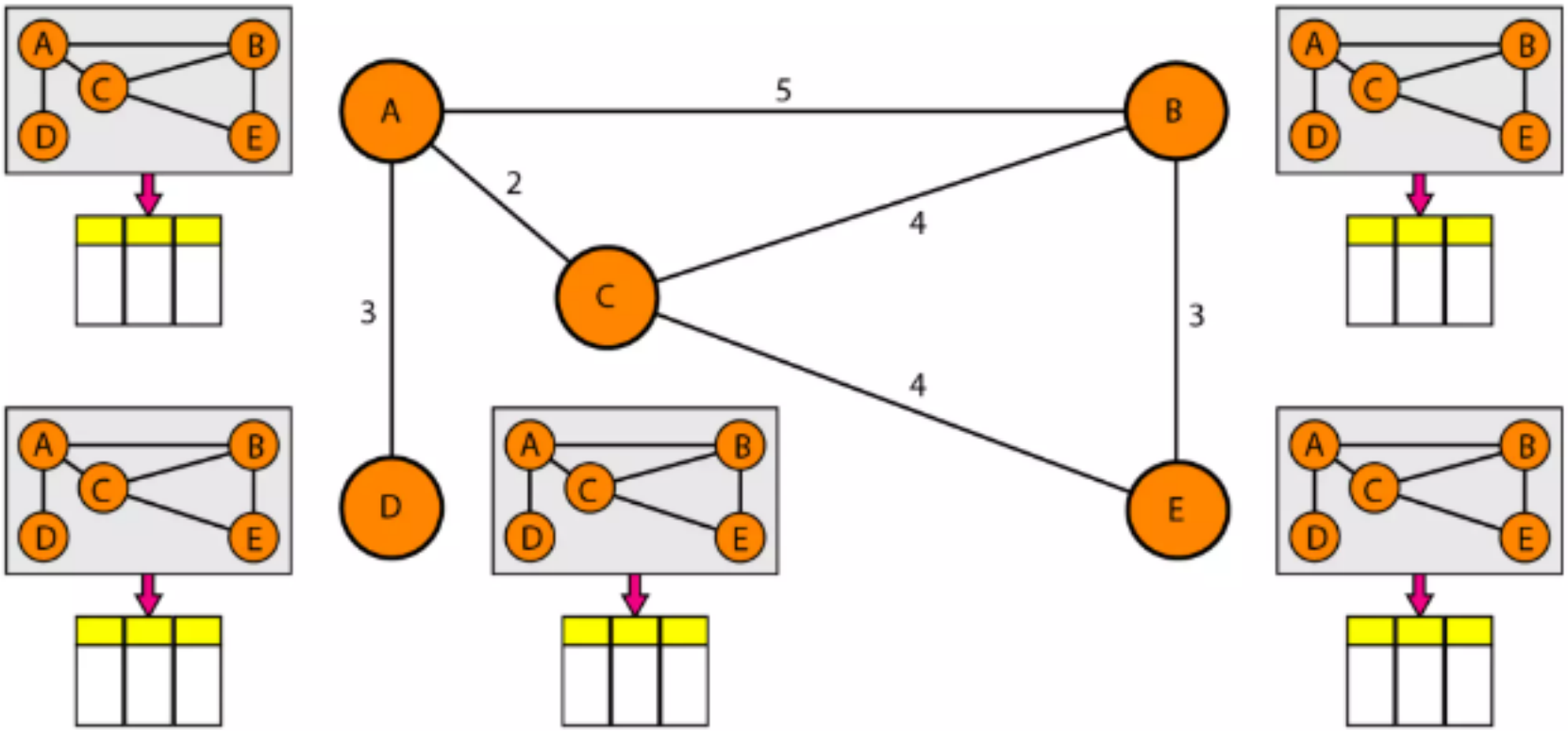
- The table of each router is also shown.
- Let us look at the routing table for R1.
- The table has seven entries to show how to reach each network in the autonomous system. Router Rl is directly connected to networks 130.10.0.0 and 130.11.0.0, which means that there are no next-hop entries for these two networks. To send a packet to one of the three networks at the far left, router R1 needs to deliver the packet to R2. The next-node entry for these three networks is the interface of router R2 with IP address 130.10.0.1. To send a packet
- to the two networks at the far right, router Rl needs to send the packet to the interface of
- router R4 with IP address 130.11.0.1. The other tables can be explained similarly.

Link State Routing

- Link state routing has a different philosophy from that of distance vector routing.
- In link state routing, **if each node in the domain has the entire topology of the domain** the list of nodes and links, how they are connected including the type, cost (metric), and condition of the links (up or down)-the node can use Dijkstra's algorithm to build a routing table.
- The topology must be dynamic, representing the latest state of each node and each link. If there are changes in any point in the network (a link is down, for example), the topology must be updated for each node.

Link State Routing

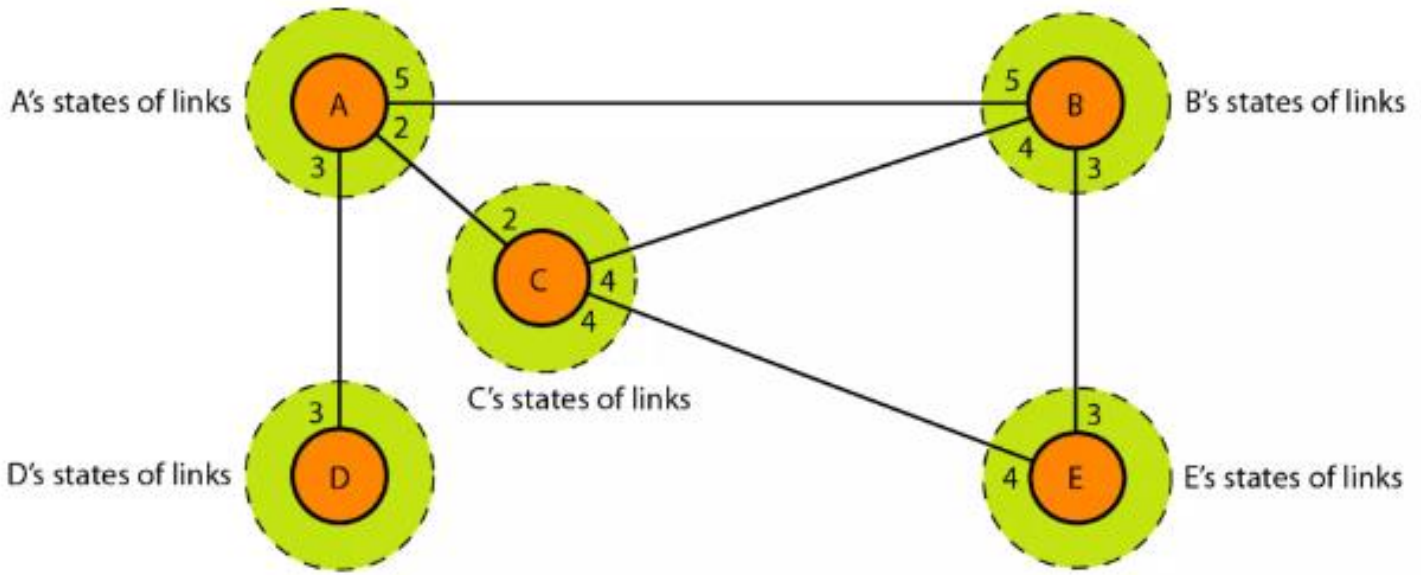
Figure 22.20 *Concept of link state routing*



Link State Routing

Link state routing is based on the assumption that, although the global knowledge about the topology is not clear, each node has partial knowledge: it knows the state (type, condition, and cost) of its links. In other words, the **whole topology can be compiled from the partial knowledge of each node.**

Figure 22.21 *Link state knowledge*



Link State Routing

Building Routing Tables

- In link state routing, **four sets of actions** are required to ensure that each node has the routing table showing the least-cost node to every other node.

1. **Creation** of the states of the links by each node, called the link state packet (LSP).
2. **Dissemination of LSPs** to every other router, called **flooding**, in an efficient and reliable way.
3. **Formation** of a shortest path tree for each node.
4. Calculation of a **routing table based** on the shortest path tree.

Link State Routing

Creation of Link State Packet (LSP)

- A link state packet can carry a **minimum amount of data: the node identity, the list of links, a sequence number, and age**.
- The **first two**, node identity and the list of links, are needed to **make the topology**.
- The **third**, sequence number, facilitates flooding and **distinguishes new LSPs from old ones**.
- The fourth, **age**, **prevents old LSPs from remaining in the domain for a long time**

Link State Routing

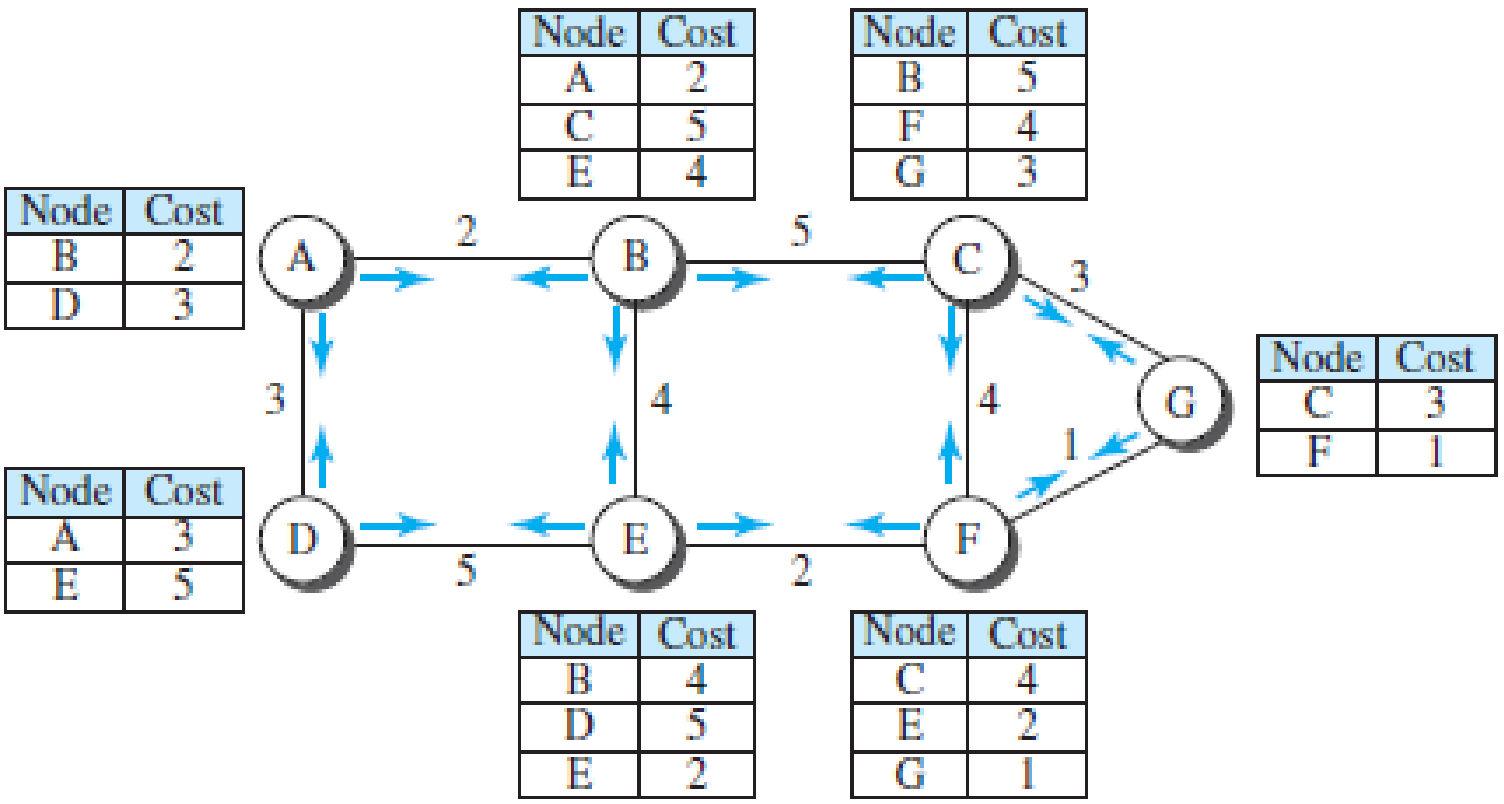
- **LSPs are generated on two occasions:**
  1. When there is a change in the topology of the domain. **Triggering** of LSP dissemination is the main way of quickly informing any node in the domain to update its topology.
  2. **On a periodic basis**: The period in this case is much longer compared to distance vector routing. The timer set for periodic dissemination is normally in the **range of 60 min or 2 h** based on the implementation. A longer period **ensures that flooding does not create too much traffic on the network**.

Link State Routing

Phase 1: Flooding of LSPs

- After a node has prepared an LSP, it must be disseminated to all other nodes, not only to its neighbors. The process is called **flooding** and based on the following:
  1. **The creating node sends a copy of the LSP out of each interface**.
  2. A node that receives an LSP **compares** it with the copy it may already have. If the newly **arrived LSP is older** than the one it has (found by checking the sequence number), it **discards** the LSP. If it is **newer, the node does the following**:
     a. It **discards the old LSP** and keeps the new one.
     b. It **sends a copy of it out of each interface** except the one from which the packet arrived. This guarantees that **flooding stops** somewhere in the domain (where a node has only one interface).

Link State Routing



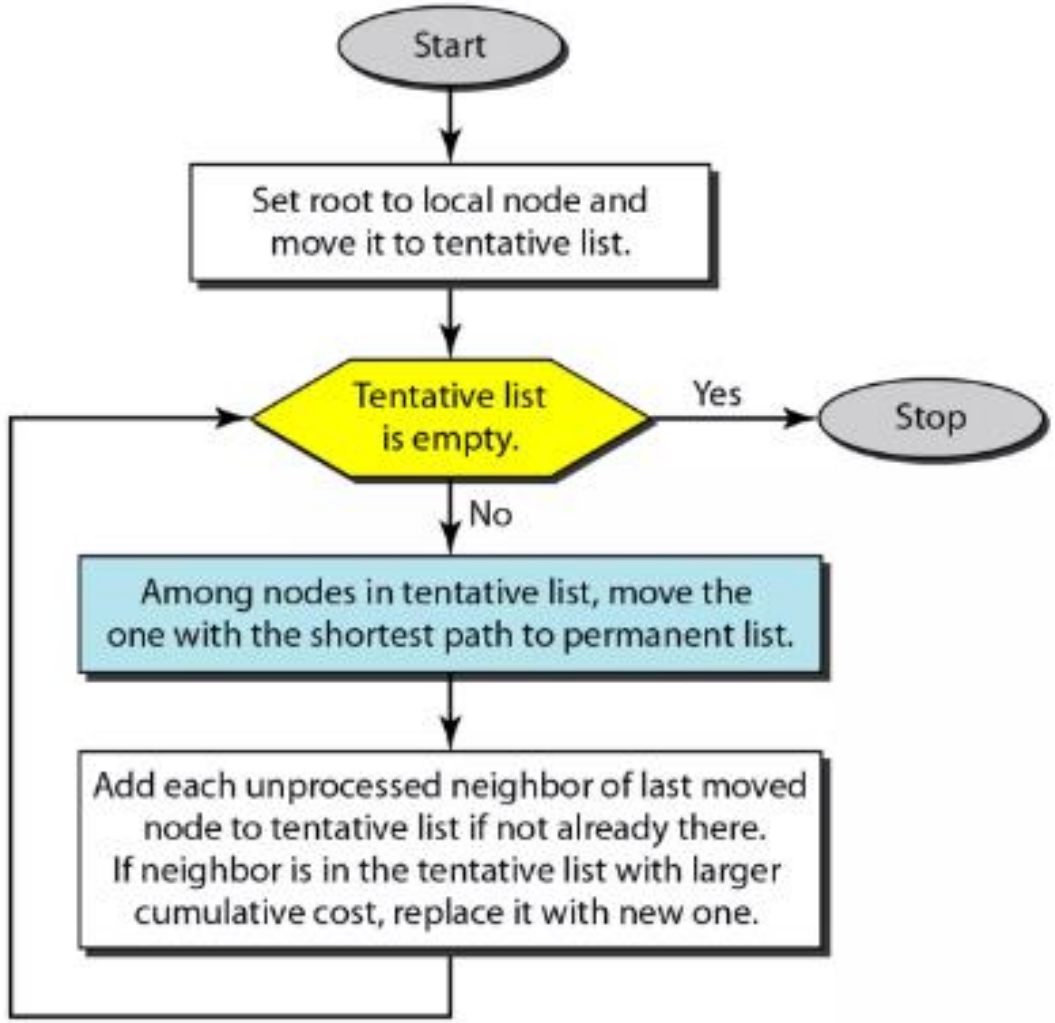
Link State Routing

Phase 2: Formation of Shortest Path Tree: Dijkstra Algorithm

- After receiving all LSPs, each node will have a copy of the whole topology.
- The topology is not sufficient to find the shortest path to every other node; a **shortest path tree is needed**.
- A tree is a graph of nodes and links; one node is called the **root**.
- **All other nodes can be reached from the root through only one single route.**
- **The Dijkstra algorithm** creates a shortest path tree from a graph.
- The algorithm divides the nodes into two sets: **tentative and permanent**.
- It finds the neighbors of a current node, makes them tentative, examines them, and if they pass the criteria, makes them permanent.

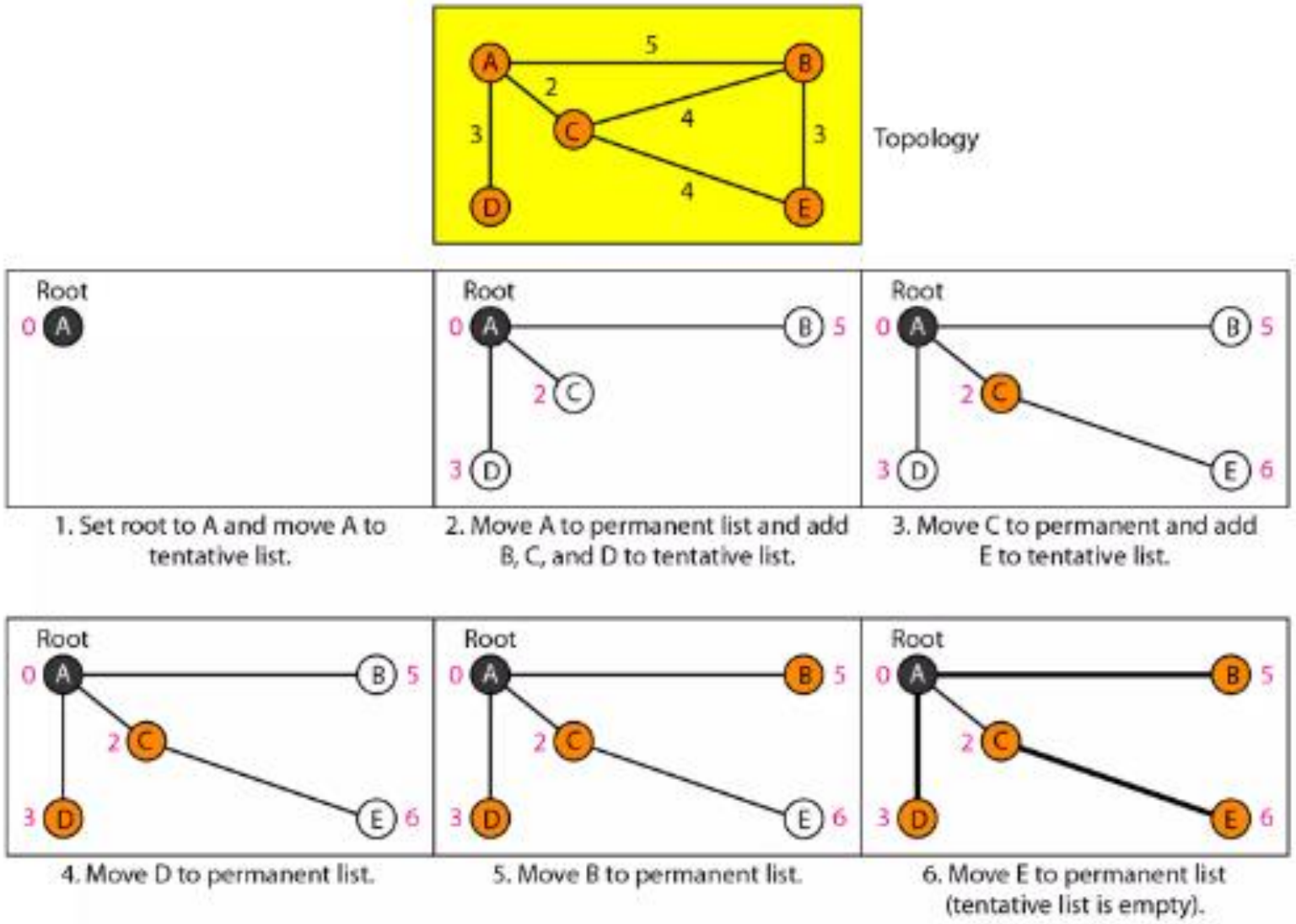
Link State Routing

Figure 22.22 *Dijkstra algorithm*



Link State Routing

Figure 22.23 *Example of formation of shortest path tree*



Li

```
Dijkstra's Algorithm ( )
{
    // Initialization
    Tree = {root}                          // Tree is made only of the root
    for (y = 1 to N)                       // N is the number of nodes
    {
        if (y is the root)
            D[y] = 0                       // D[y] is shortest distance from root to node y
        else if (y is a neighbor)
            D[y] = c[root][y]              // c[x][y] is cost between nodes x and y in LSDB
        else
            D[y] = ∞
    }
    // Calculation
    repeat
    {
        find a node w, with D[w] minimum among all nodes not in the Tree
        Tree = Tree ∪ {w}                  // Add w to tree
        // Update distances for all neighbors of w
        for (every node x, which is a neighbor of w and not in the Tree)
        {
            D[x] = min{D[x], (D[w] + c[w][x])}
        }
    } until (all nodes included in the Tree)
} // End of Dijkstra
```

Link State Routing

1. We make node A the root of the tree and move it to the tentative list. Our two lists are

   Permanent list: empty Tentative list: A(O)

2. Node A has the shortest cumulative cost from all nodes in the tentative list. We move A to the pennanent list and add all neighbors of A to the tentative list. Our new lists are

   Permanent list: A(O) Tentative list: B(5), C(2), D(3)

3. Node C has the shortest cumulative cost from all nodes in the tentative list. We move C to the permanent list. Node C has three neighbors, but node A is already processed, which makes the unprocessed neighbors just B and E. However, B is already in the tentative list with a cumulative cost of 5. Node A could also reach node B through C with a cumulative cost of 6. Since 5 is less than 6, we keep node B with a cumulative cost of 5 in the tentative list and do not replace it. Our new lists are

   Permanent list: A(O), e(2) Tentative list: B(5), 0(3), E(6)

Link State Routing

4. Node D has the shortest cumulative cost of all the nodes in the tentative list. We move D to the permanent list. Node D has no unprocessed neighbor to be added to the tentative list. Our new lists are

   Permanent list: A(O), C(2), 0(3) Tentative list: B(5), E(6)

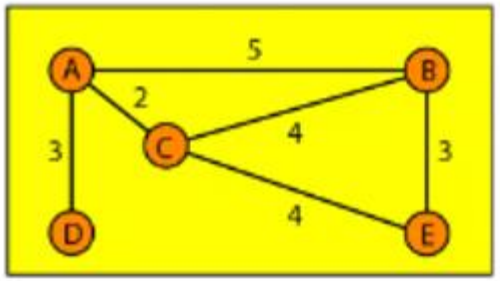
5. Node B has the shortest cumulative cost of all the nodes in the tentative list. We move B to the permanent list. We need to add all unprocessed neighbors of B to the tentative list (this is just node E). However, E(6) is already in the list with a smaller cumulative cost. The cumulative cost to node E, as the neighbor of B, is 8. We keep node E(6) in the tentative list. Our new lists are

   Permanent list: A(O), B(5), C(2), 0(3) Tentative list: E(6)

6. Node E has the shortest cumulative cost from all nodes in the tentative list. We move E to the permanent list. Node E has no neighbor. Now the tentative list is empty. We stop; our shortest path tree is ready. The final lists are

   Permanent list: A(0), B(5), C(2), D(3), E(6) Tentative list: empty

Link State Routing

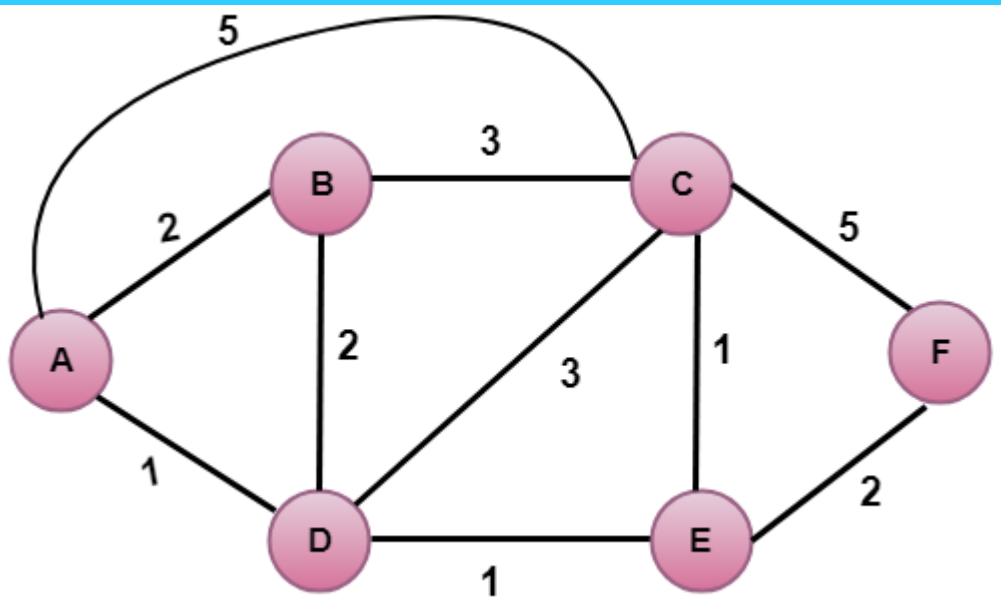


Topology

Table 22.2 *Routing table for node A*

Node	*Cost*	*Next Router*
A	0	-
B	5	-
C	2	-
D	3	-
E	6	C

Link State Routing

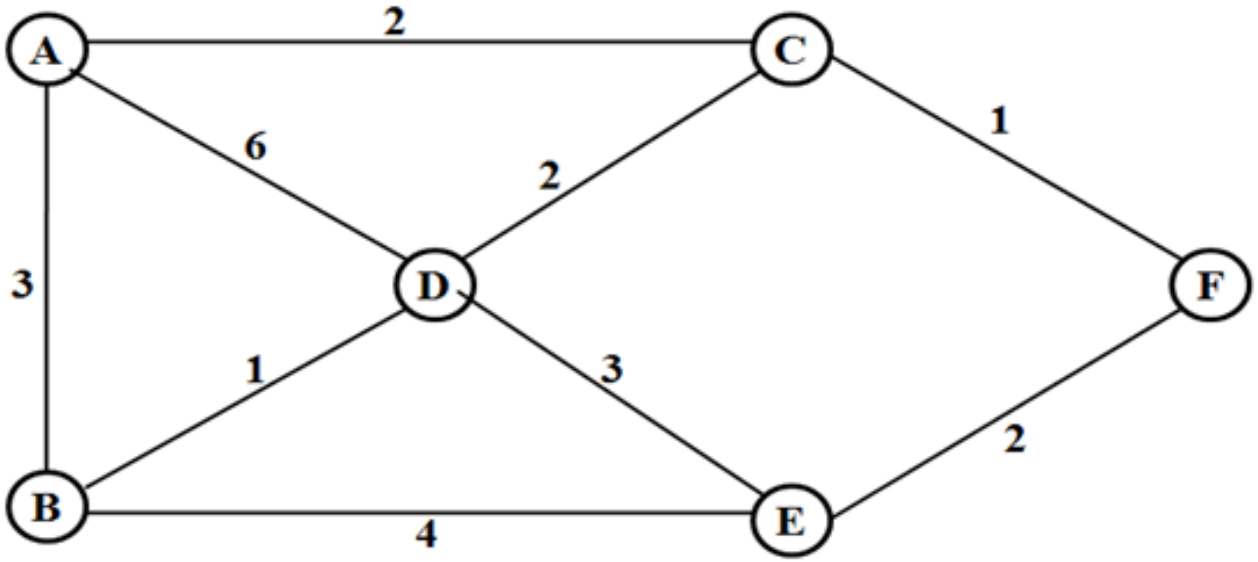


Final table:

Step	N	D(B),P(B)	D(C),P(C)	D(D),P(D)	D(E),P(E)	D(F),P(F)
1	A	2,A	5,A	1,A	∞	∞
2	AD	2,A	4,D		2,D	∞
3	ADE	2,A	3,E			4,E
4	ADEB		3,E			4,E
5	ADEBC					4,E
6	ADEBCF					

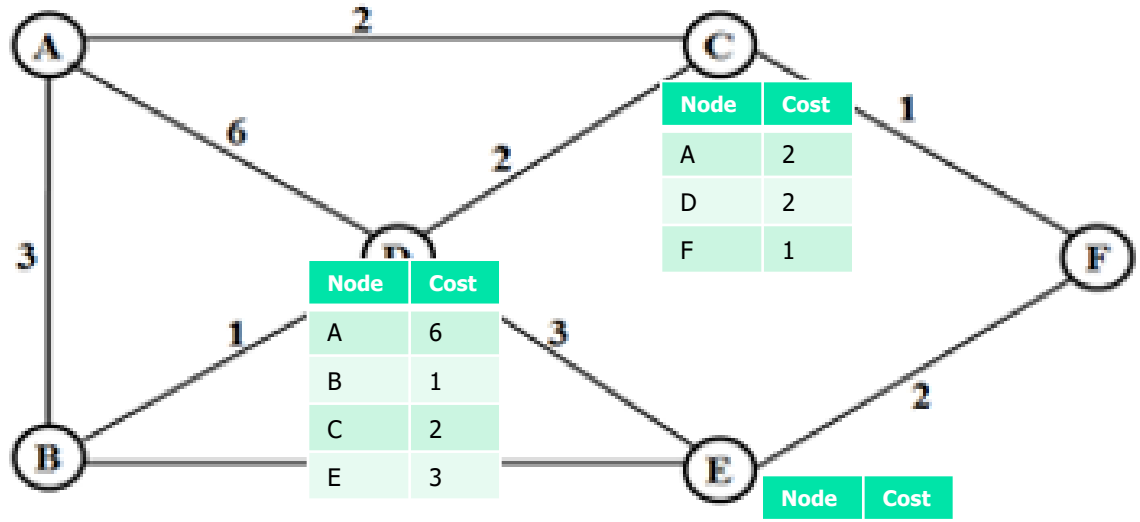
Link State Routing

Calculate the shortest path from the root node A to all other nodes in the following network configuration using Link State Routing.



Link State Routing

Calculate the shortest path from the root node A to all other nodes in the following network configuration using Link State Routing.



Node	Cost
B	3
C	2
D	6

Node	Cost
A	2
D	2
F	1

Node	Cost
C	1
E	2

Node	Cost
A	6
B	1
C	2
E	3

Node	Cost
A	3
D	1
E	4

Node	Cost
B	4
D	3
F	2

Link State Routing

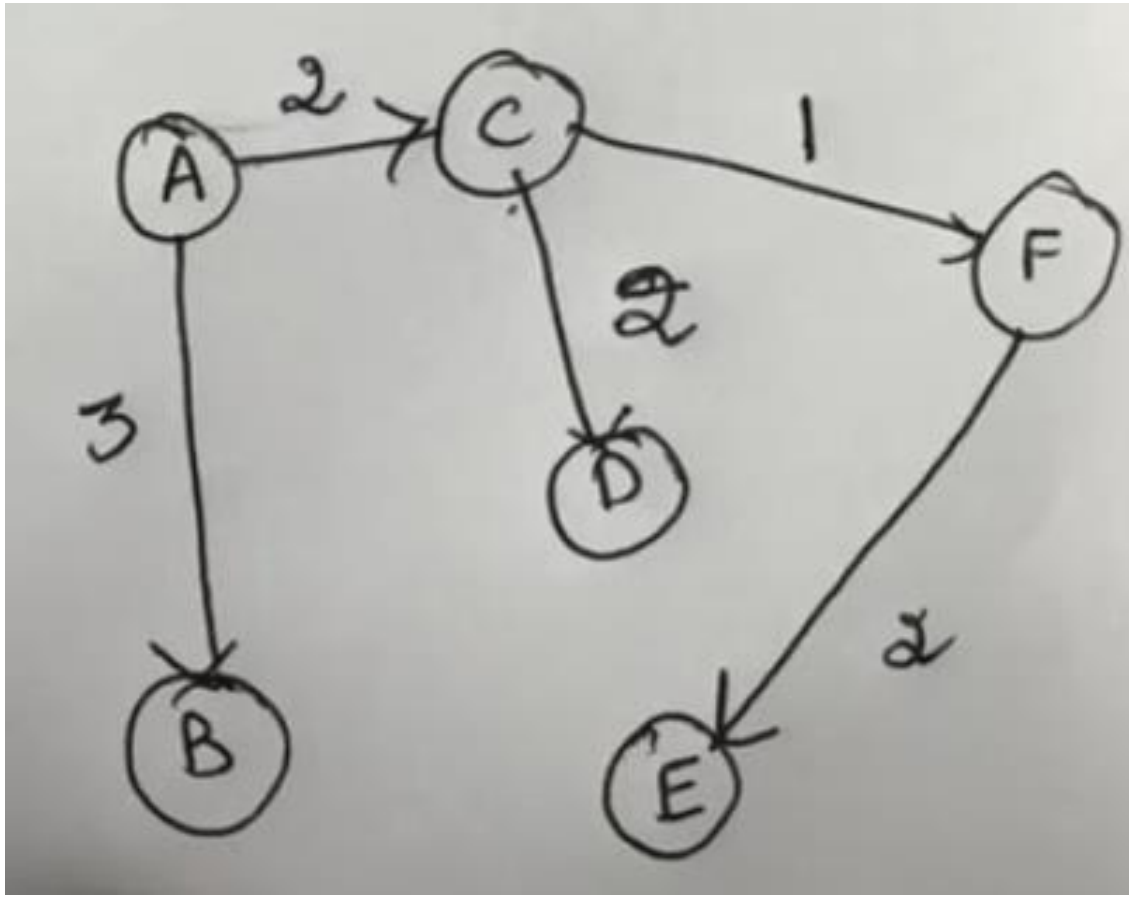
Step	N	D(B),P(B)	D(C),P(C)	D(D),P(D)	D(E),P(E)	D(F),P(F)

Li

#dijkstra Link State Routing -Dijkstra's Algorithm -Computer Networks

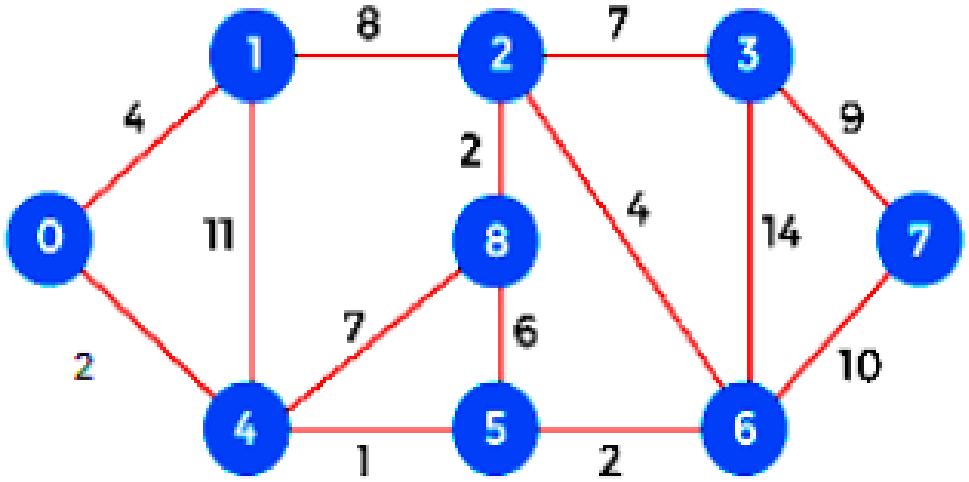
Iteration	Tree	B	C	D	E	F
Initial	{A}	3	(2)	5	∞	∞
1	{A, C}	(3)	-	4	∞	3
2	{A, B, C}	-	-	4	7	(3)
3	{A, B, C, F}	-	-	(4)	5	-
4	{A, B, C, D, F}	-	-	-	5	-
5	{A, B, C, D, E, F}	-	-	-	-	-

Link State Routing



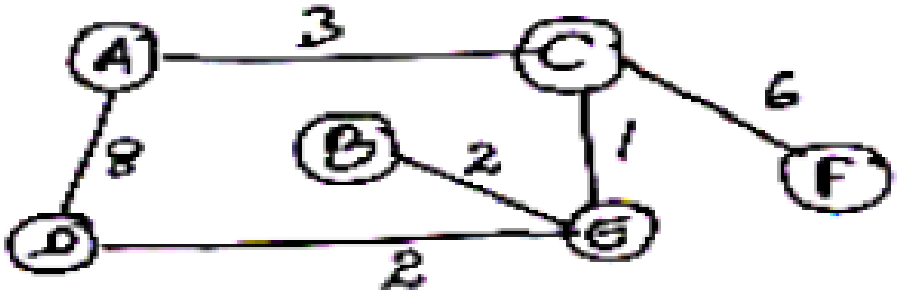
Link State Routing

Calculate the shortest path from source node (Node 1) to all other nodes in the network using Dijkstra's algorithm and draw the shortest path tree for the same.



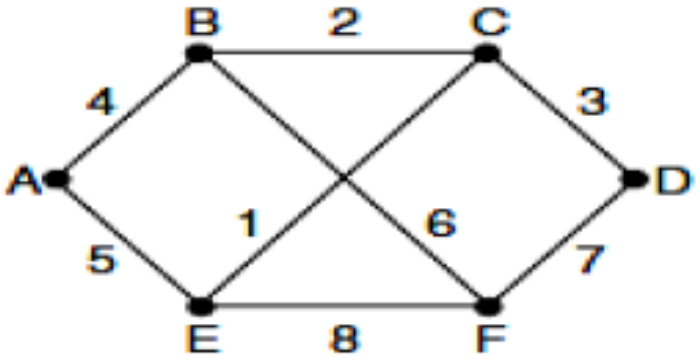
Link State Routing

Use the Dijkstra's algorithm and find the shortest path from node A to all other nodes. Clearly give the shortest path trees for all the iterations and final shortest path table:



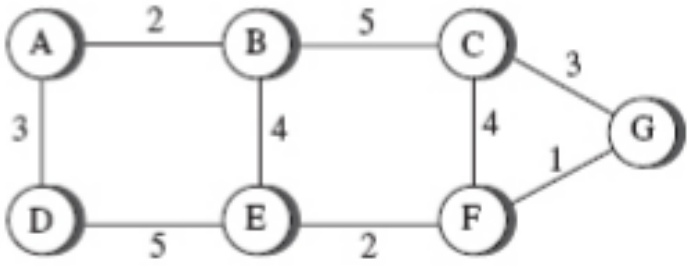
Link State Routing

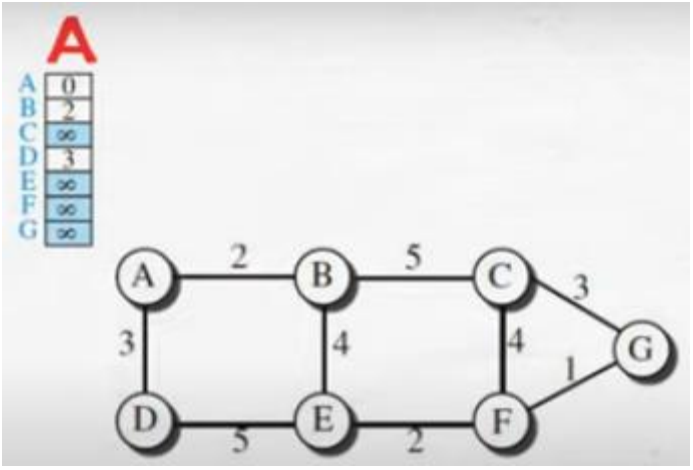
Consider the network. Distance vector routing is used, and the following vectors have just come in to router C: from B: (5, 0, 8, 12, 6, 2); from D: (16, 12, 6, 0, 9, 10); and from E: (7, 6, 3, 9, 0, 4). The cost of the links from C to B, D, and E, are 6, 3, and 5, respectively. What is C's new routing table? Give both the outgoing line to use and the cost. CO2 (06)



Link State Routing

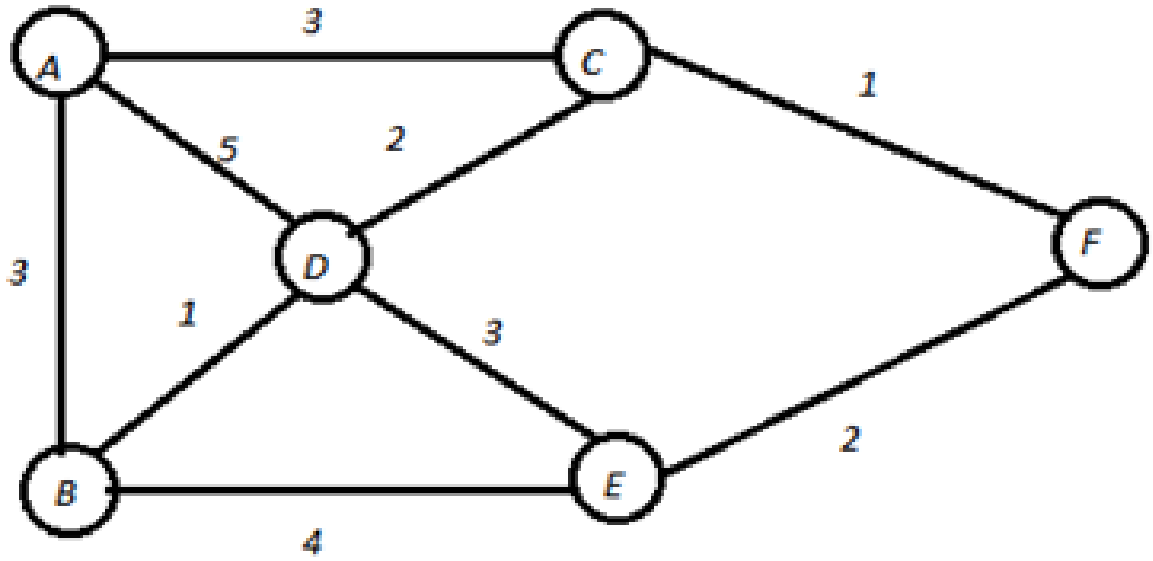
Draw the forwarding table for all nodes and show the updated vectors using bellman ford algorithm for node B after receiving the vectors from node A and E. Create the tree and distance vector for node B. CO3 (06)





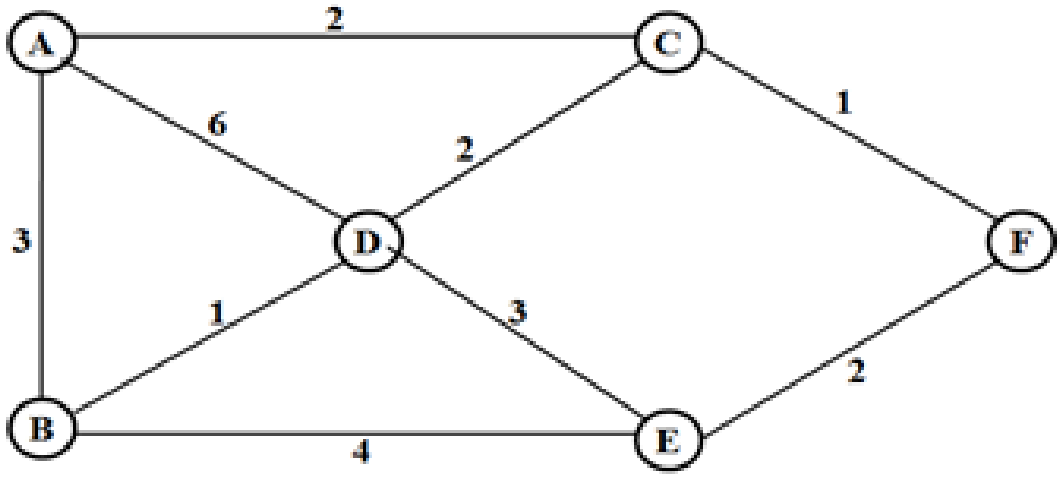
Link State Routing

Consider the network given below. Use Dijkstra's algorithm to find the shortest path from the source node E to all the other destination nodes. Find the shortest path tree from node E to the other nodes. CO2 (10)

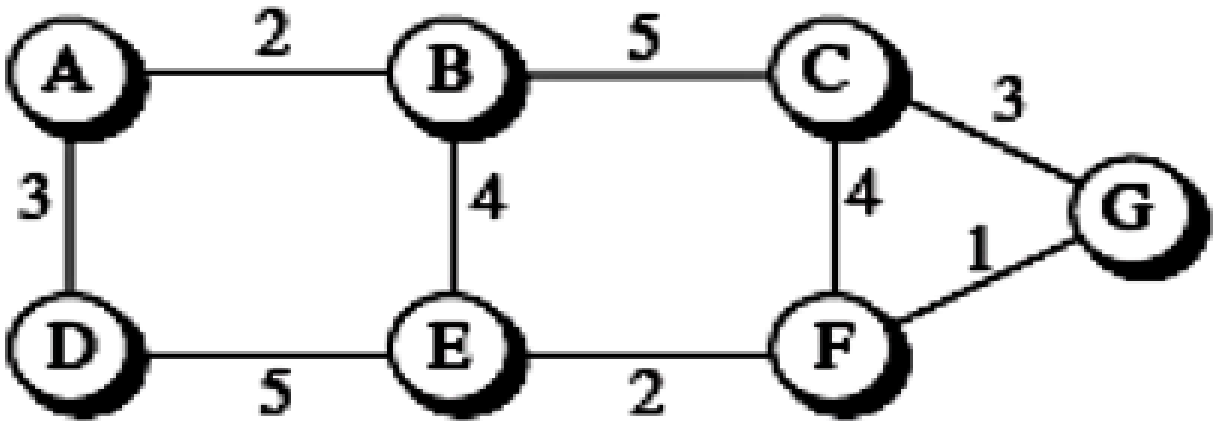


Link State Routing

Calculate the shortest path from the root node A to all other nodes in the following network configuration using Link State Routing. CO2 (10)



Create the shortest-path tree and the forwarding table for node B in the network given below using Dijkstra's routing algorithm. CO3 (06)



Link State Routing

S.No.	Distance Vector Routing	Link State Routing
1.	Bandwidth required is less due to local sharing, small packets and no flooding.	Bandwidth required is more due to flooding and sending of large link state packets.
2.	Based on local knowledge, since it updates table based on information from neighbours.	Based on global knowledge, it have knowledge about entire network.
3.	Make use of Bellman Ford Algorithm.	Make use of Dijakstra's algorithm.
4.	Traffic is less.	Traffic is more.
5.	Converges slowly i.e, good news spread fast and bad news spread slowly.	Converges faster.
6.	Count of infinity problem.	No count of infinity problem.

Path Vector Routing

Path Vector Routing

- Distance vector and link state routing are used inside an autonomous system, but not between autonomous systems. They are **not suitable for interdomain** routing mostly because of scalability.
- **Distance vector** routing is subject to **instability** if there are more than a few hops in the domain of operation.
- **Link state routing** needs a **huge amount of resources to calculate routing tables.** It also creates heavy traffic because of flooding. There is a need for a third routing protocol which we call path vector routing.

Path Vector Routing

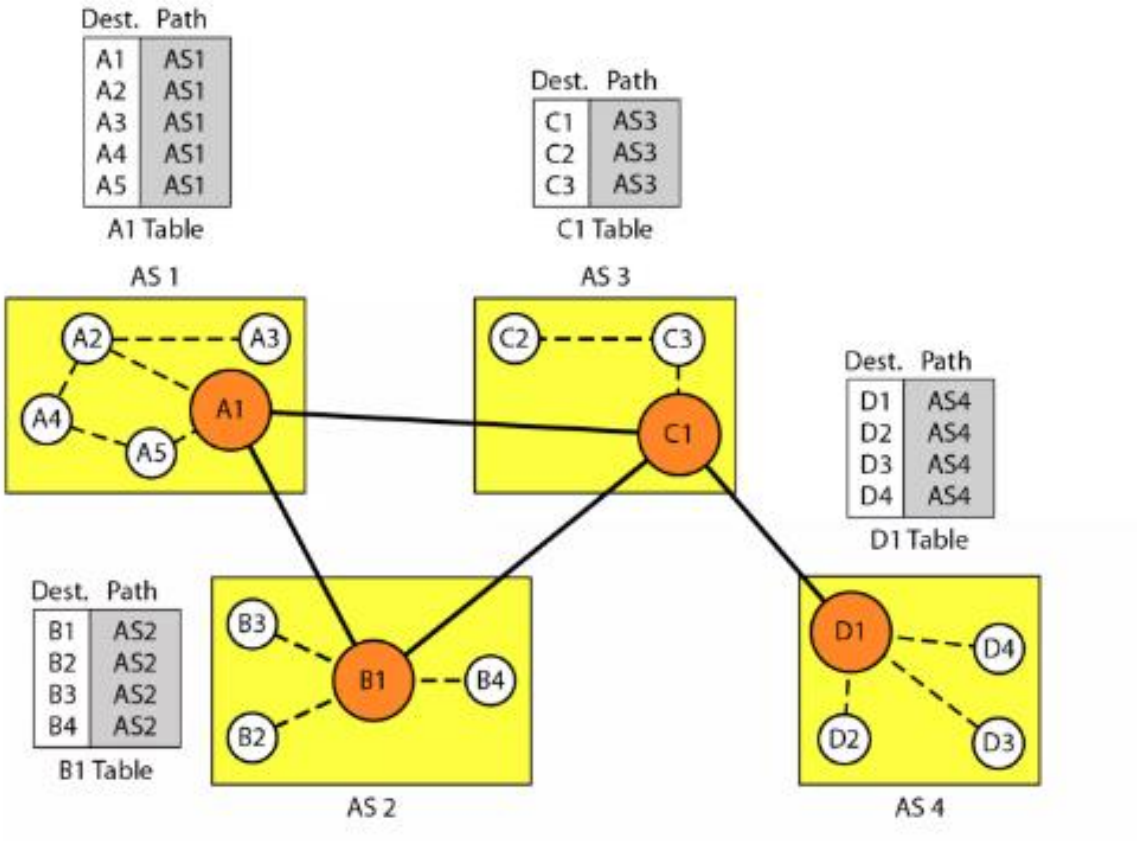
- The principle of path vector routing is similar to that of distance vector routing.
- One node in each autonomous system that acts on behalf of the entire autonomous system. It is called as the **speaker node**.
- The speaker node in an AS creates a **routing table** and advertises it to speaker nodes in the neighboring ASs.
- A speaker node **advertises the path, not the metric of the nodes**, in its autonomous system or other autonomous systems.

Path Vector Routing

Initialization

At the beginning, each speaker node can know only the **reachability of nodes inside its autonomous system**. Figure 22.30 shows the initial tables for each speaker node in a system made of four ASs.

Figure 22.30 *Initial routing tables in path vector routing*

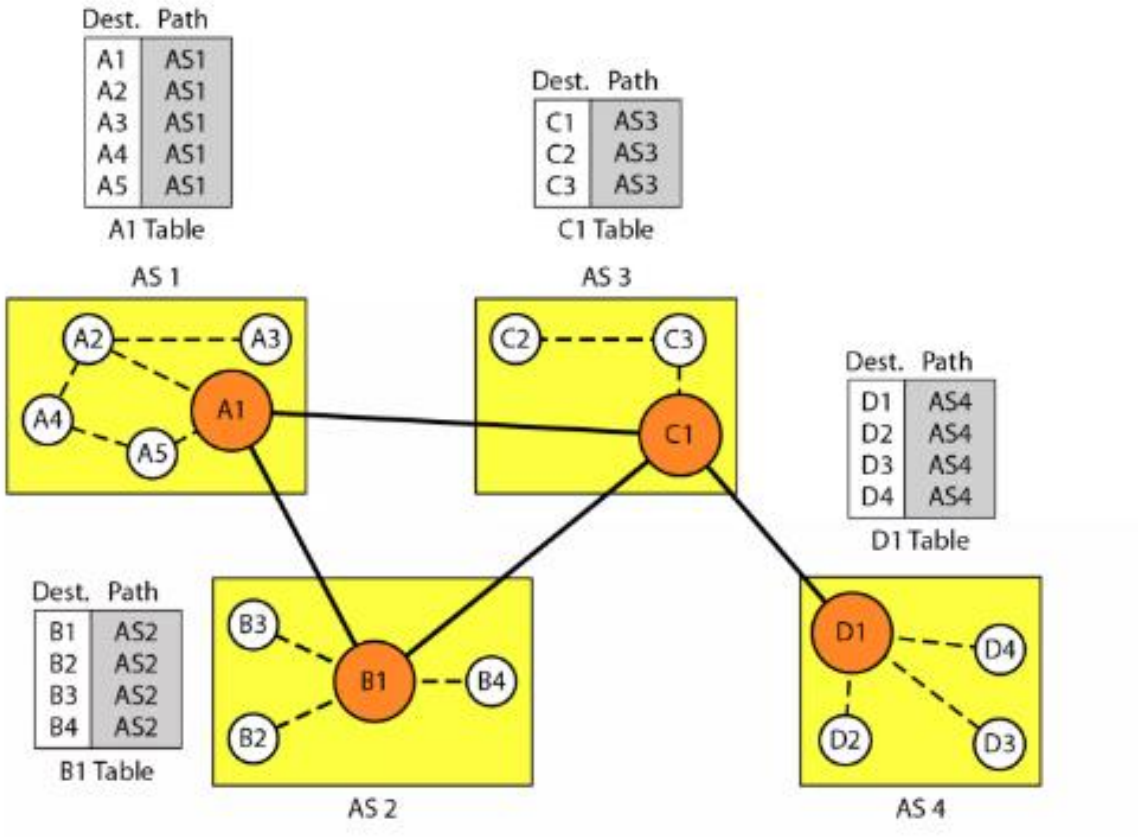


Path Vector Routing

Sharing

A speaker in an autonomous system shares its table with immediate neighbors. In Figure 22.30, node A1 shares its table with nodes B1 and C1. Node C1 shares its table with nodes D1, B1, and A1. Node B1 shares its table with C1 and A1. Node D1 shares its table with C1.

Figure 22.30 *Initial routing tables in path vector routing*



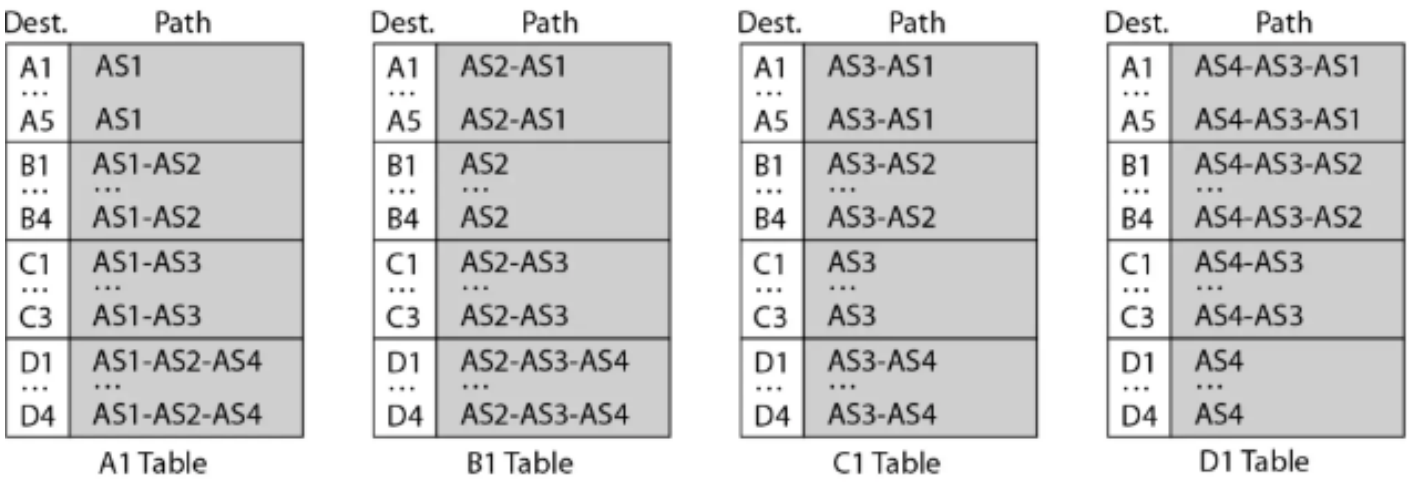
Path Vector Routing

Updating

When a speaker node receives a two-column table from a neighbor, it updates its own table by adding the nodes that are not in its routing table and adding its own autonomous system and the autonomous system that sent the table.

After a while each speaker has a table and knows how to reach each node in other ASs. Figure 22.31 shows the tables for each speaker node after the **system is stabilized**.

Figure 22.31 *Stabilized tables for three autonomous systems*

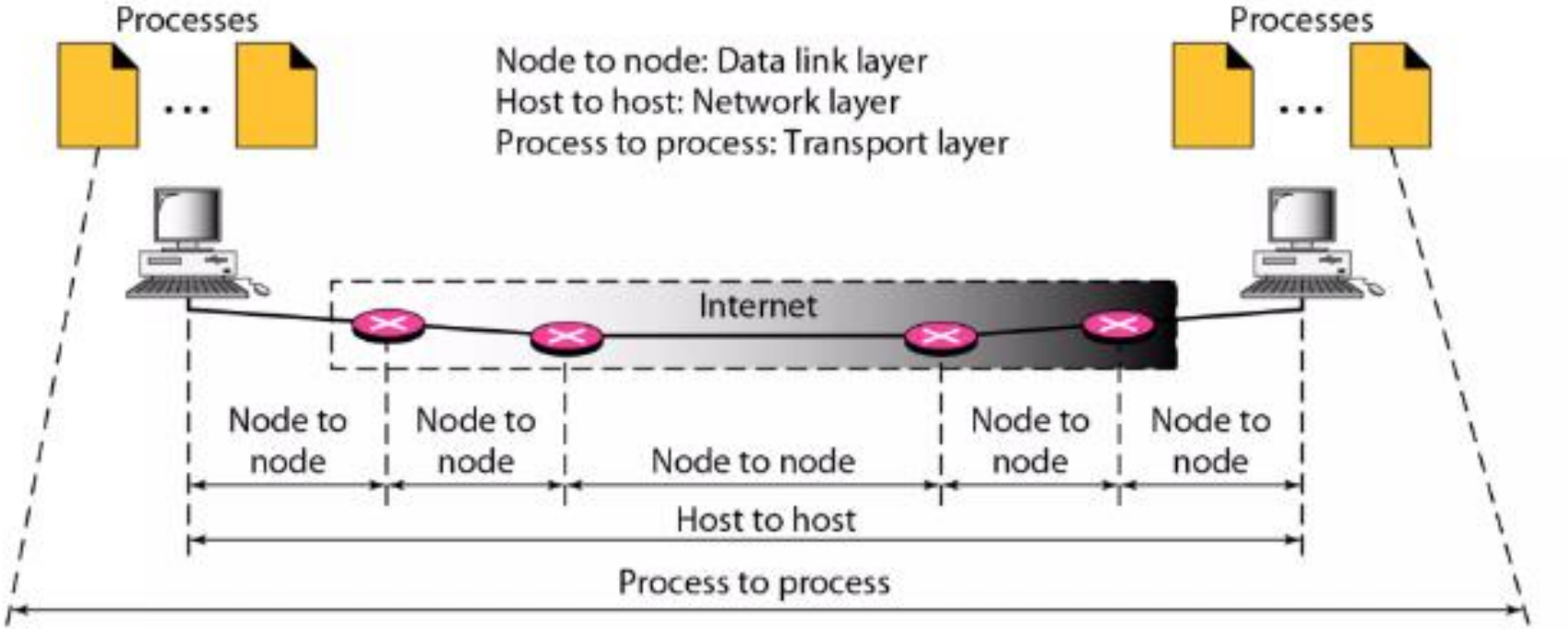


Transport Layer: PROCESS-TO-PROCESS DELIVERY

- The **data link layer** is responsible for delivery of frames between two neighboring nodes over a link. This is called **node-to-node delivery**.
- The **network layer** is responsible for delivery of datagrams between two hosts. This is called **host-to-host delivery**.
- Real Communication on the internet takes place between two processes (application programs). This is called **process-to-process delivery**.
- **Several processes** may be running on the source host and several on the destination host. To complete the delivery, we need a mechanism to deliver data from one of these processes running on the source host to the corresponding process running on the destination host.
- The transport layer is responsible for process-to-process delivery-the delivery of a packet, part of a message, from one process to another. Two processes communicate in a client/server relationship. Figure 23.1 shows these three types of deliveries and their domains.

Transport Layer: PROCESS-TO-PROCESS DELIVERY

Figure 23.1 *Types of data deliveries*



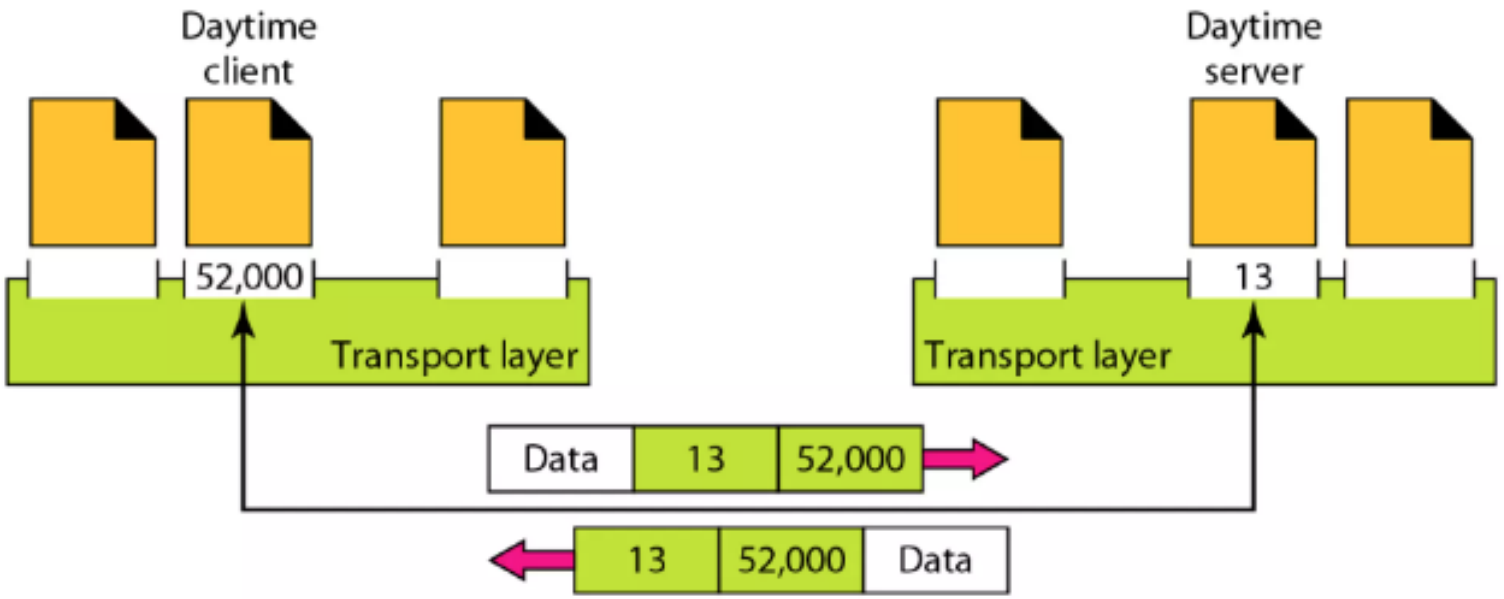
Transport Layer: Port Addressing

- At the transport layer, we need a transport layer address, called a **port number**, to choose among multiple processes running on the destination host.
- The destination port number is needed for **delivery**; the source port number is needed for the **reply**.
- In the Internet model, the **port numbers are 16-bit integers between 0 and 65,535.**
- The **client program** defines itself with a port number, chosen randomly by the transport layer software running on the client host. This is the **ephemeral port number**.
- The **Internet** uses universal port numbers for servers; these are called **well-known port numbers**.

Transport Layer: PROCESS-TO-PROCESS DELIVERY

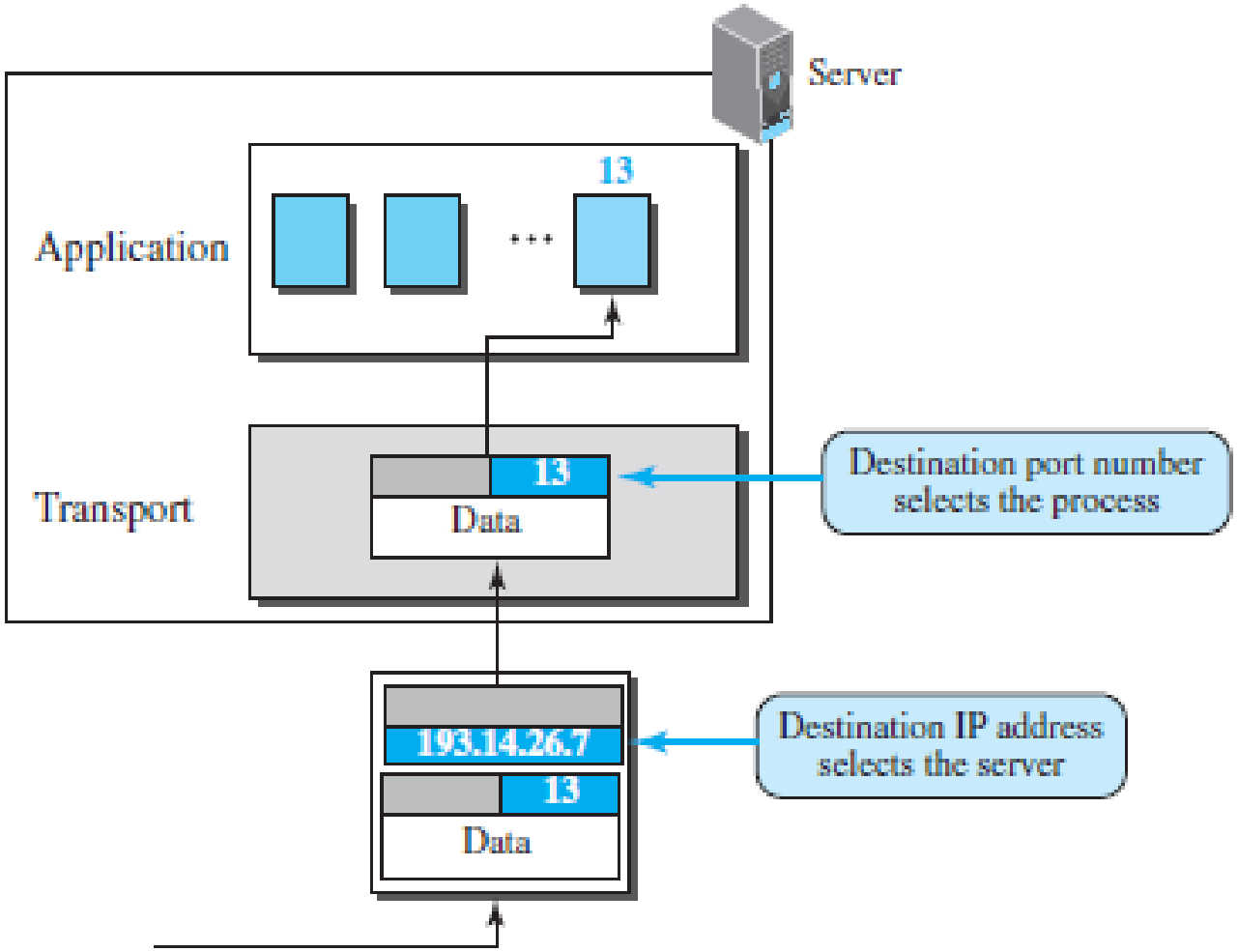
- For example, while the Daytime client process, can use an ephemeral (temporary) port number 52,000 to identify itself, the Daytime server process must use the well-known (permanent) port number 13. Figure 23.2 shows this concept.

Figure 23.2 *Port numbers*



Transport Layer: PROCESS-TO-PROCESS DELIVERY

Figure 23.4 *IP addresses versus port numbers*

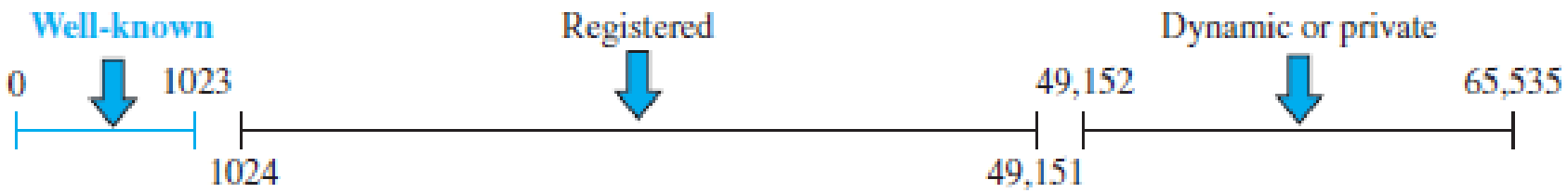


Transport Layer: PROCESS-TO-PROCESS DELIVERY

ICANN (Internet Corporation for Assigned Names and Numbers) Ranges

- ICANN has divided the port numbers into three ranges: well-known, registered, and dynamic (or private), as shown in Figure 23.5.

Figure 23.5 *ICANN ranges*



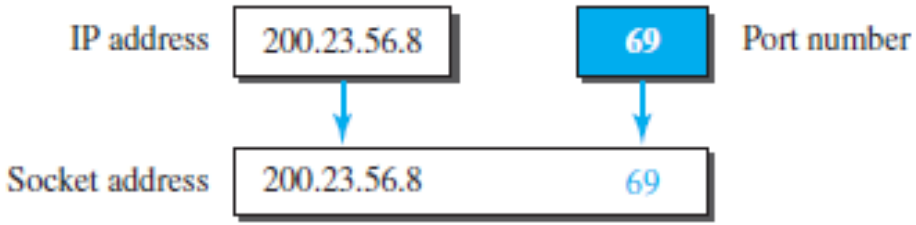
- **Well-known ports**. The ports ranging from 0 to 1023 are assigned and controlled by ICANN. These are the well-known ports.
- **Registered ports**. The ports ranging from 1024 to 49,151 are not assigned or controlled by ICANN. They can only be registered with ICANN to prevent duplication.
- **Dynamic ports**. The ports ranging from 49,152 to 65,535 are neither controlled nor registered. They can be used as temporary or private port numbers.

Transport Layer: PROCESS-TO-PROCESS DELIVERY

Socket Addresses

- A transport-layer protocol in the TCP suite needs both the IP address and the port number, at each end, to **make a connection**.
- The combination of an IP address and a port number is called a **socket address.** The client socket address defines the client process uniquely just as the server socket address defines the server process uniquely

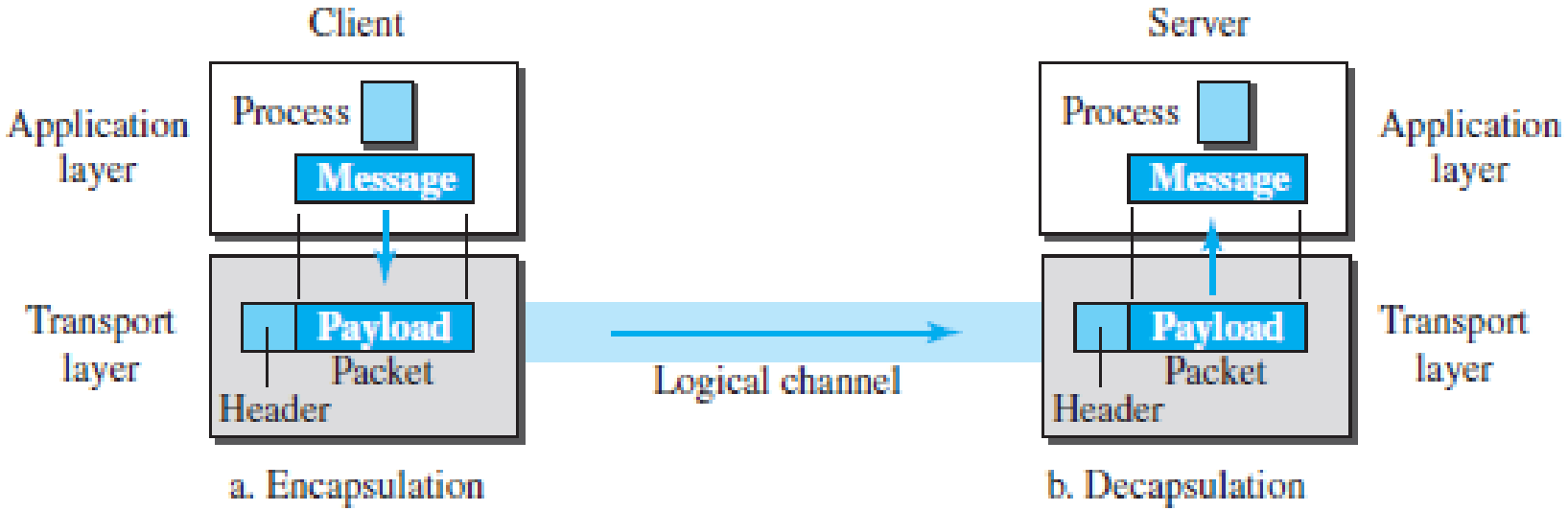
Figure 23.6 *Socket address*



Transport Layer: PROCESS-TO-PROCESS DELIVERY

- To send a message from one process to another, the transport-layer protocol encapsulates and decapsulates messages. Encapsulation happens at the sender site. When a process has a message to send, it passes the message to the transport layer along with a pair of socket addresses and some other pieces of information, which depend on the transport-layer protocol. The transport layer receives the data and adds the transport-layer header.
- Decapsulation happens at the receiver site. When the message arrives at the destination transport layer, the header is dropped and the transport layer delivers the message to the process running at the application layer. The sender socket address is passed to the process in case it needs to respond to the message received.

Figure 23.7 *Encapsulation and decapsulation*



Transport Layer: PROCESS-TO-PROCESS DELIVERY

Multiplexing

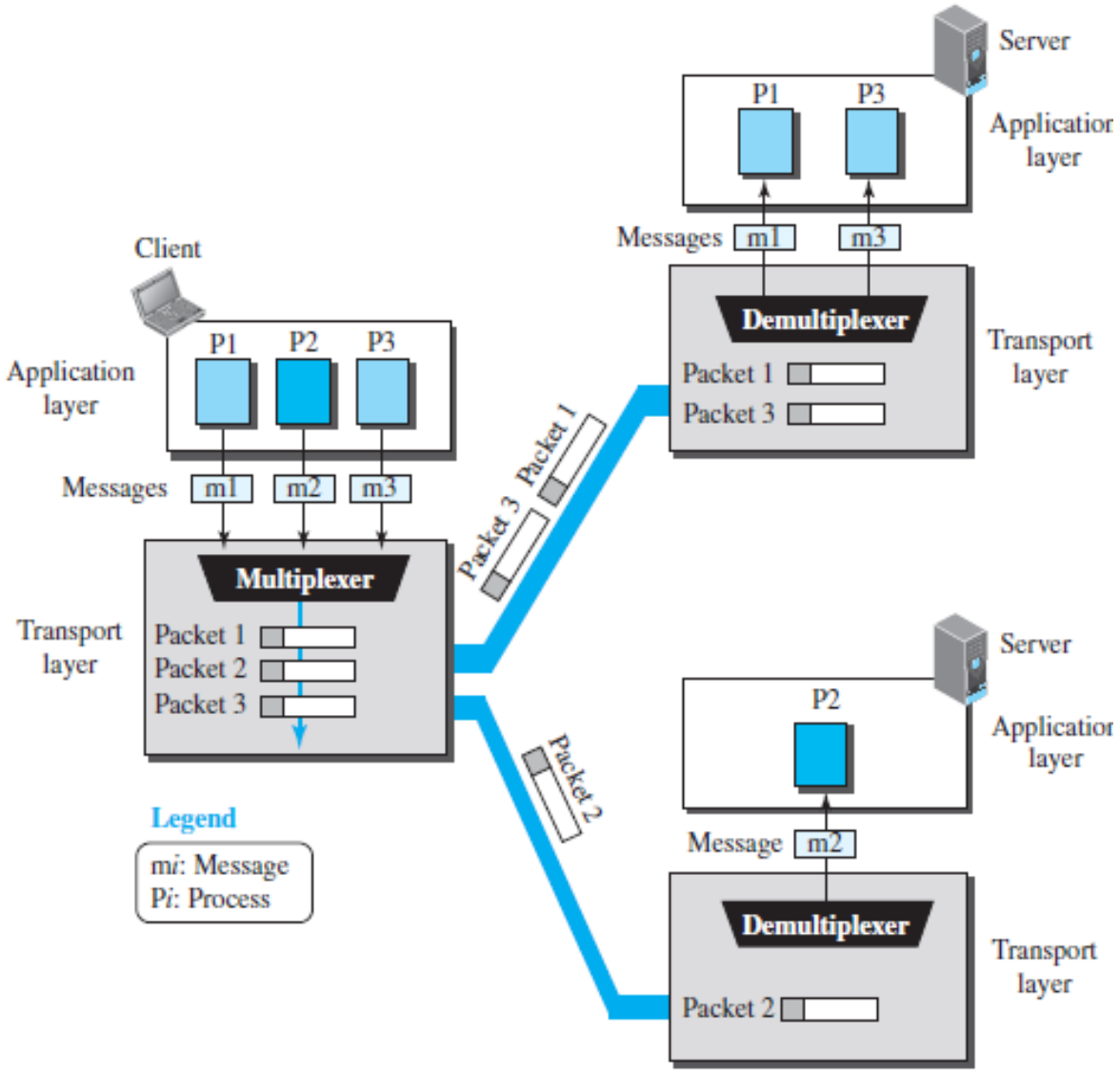
- At the sender site, there may be several processes that need to send packets.
- This is a **many-to-one relationship and requires multiplexing**.
- The protocol accepts messages from different processes, differentiated by their assigned port numbers.
- After adding the header, the transport layer passes the packet to the network layer.

Demultiplexing

- At the receiver site, the relationship is **one-to-many and requires demultiplexing.**
- The transport layer receives datagrams from the network layer. After error checking and dropping of the header, the transport layer delivers each message to the appropriate process based on the port number**.**

Transport Layer: PROCESS-TO-PROCESS DELIVERY

Figure 23.8 *Multiplexing and demultiplexing*



Transport Layer: PROCESS-TO-PROCESS DELIVERY

Connectionless Versus Connection-Oriented Service

- A transport layer protocol can either be connectionless or connection-oriented.

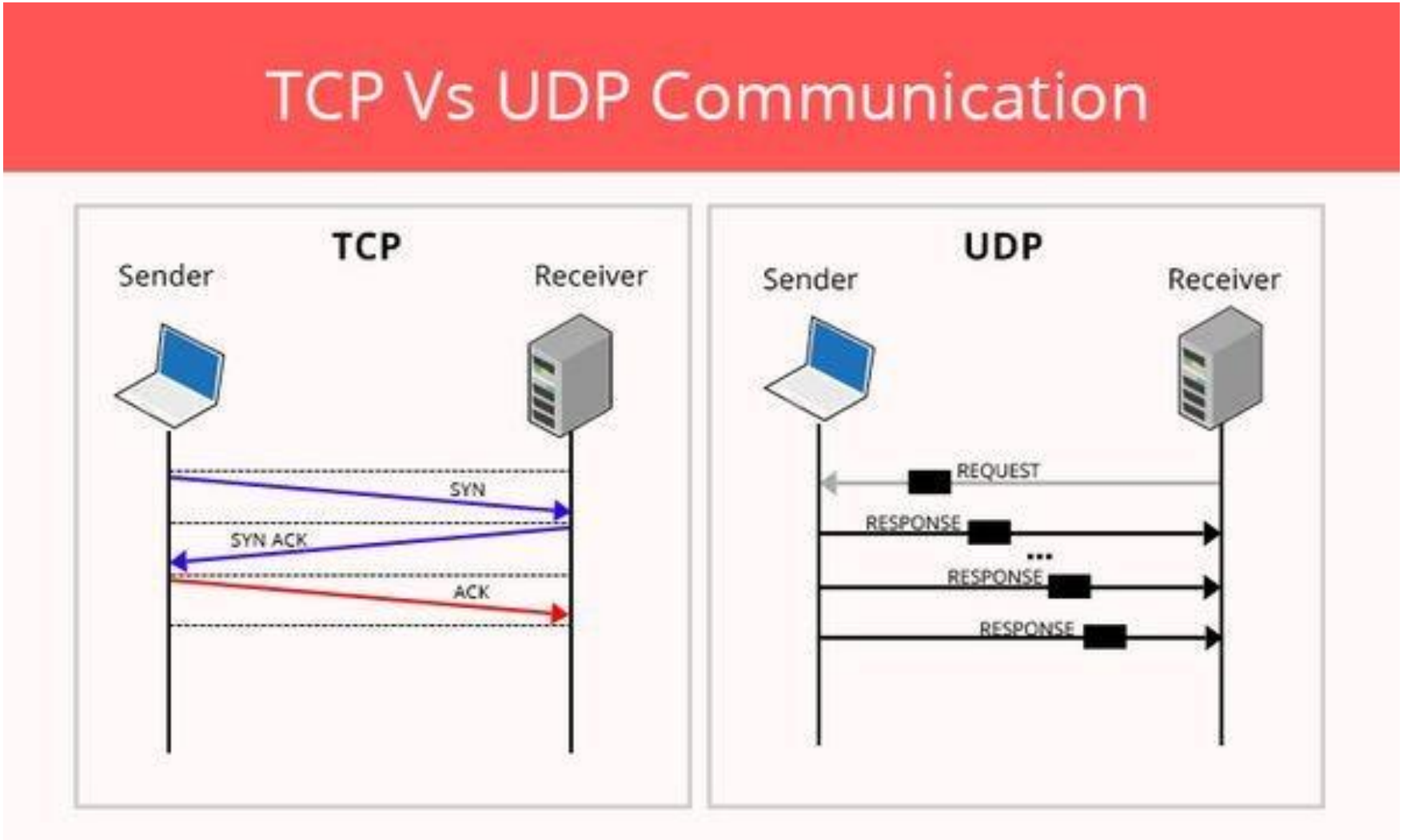
Connectionless Service

- In a connectionless service, the packets are sent from one party to another with **no need for connection establishment or connection release**.
- The packets are **not numbered**; they **may be delayed or lost or may arrive out of sequence.**
- There is **no acknowledgment** either.
- One of the transport layer protocols in the Internet model, **UDP, is connectionless.**

Connection-Oriented Service

- In a connection-oriented service, a **connection is first established** between the sender and the receiver.
- Data are **transferred**.
- At the end, the **connection is released**. TCP and SCTP are connection-oriented protocols.

Transport Layer: USER DATAGRAM PROTOCOL (UDP)



Transport Layer: PROCESS-TO-PROCESS DELIVERY

Reliable Versus Unreliable

- If the application layer program needs **reliability**, we use a reliable transport layer protocol by implementing flow and error control at the transport layer. This means a slower and more complex service.
- If the application program does **not need reliability** or it needs fast service or the nature of the service does not demand flow and error control (real-time applications), then an unreliable protocol can be used.
- In the Internet, there are **three common different transport layer protocols**
- **UDP** is connectionless and unreliable;
- **TCP and SCTP** are connection oriented and reliable.

Transport Layer: USER DATAGRAM PROTOCOL (UDP)

- The User Datagram Protocol (UDP) is called a **connectionless**, **unreliable** transport protocol, very limited error checking.

Some advantages of UDP

- It is a very simple protocol using a **minimum of overhead**.
- If a process wants to **send a small message** and does not care much about reliability, it can use UDP.
- Sending a small message by using UDP **takes much less interaction** between the sender and receiver than using TCP or SCTP.

Transport Layer: USER DATAGRAM PROTOCOL (UDP)

- Well-Known Ports for UDP

Table 23.1 *Well-known ports used with UDP*

Port	*Protocol*	*Description*
7	Echo	Echoes a received datagram back to the sender
9	Discard	Discards any datagram that is received
11	Users	Active users
13	Daytime	Returns the date and the time
17	Quote	Returns a quote of the day
19	Chargen	Returns a string of characters
53	Nameserver	Domain Name Service
67	BOOTPs	Server port to download bootstrap information
68	BOOTPc	Client port to download bootstrap information
69	TFTP	Trivial File Transfer Protocol
111	RPC	Remote Procedure Call
123	NTP	Network Time Protocol
161	SNMP	Simple Network Management Protocol
162	SNMP	Simple Network Management Protocol (trap)

Transport Layer: USER DATAGRAM PROTOCOL (UDP)

- Well-Known Ports for UDP

Port Number	Protocol	Application
20	TCP	FTP Data
21	TCP	FTP Control
22	TCP	SSH
23	TCP	Telnet
25	TCP	SMTP
53	UDP,TCP	DNS
67,68	UDP	DHCP
69	UDP	TFTP
80	TCP	HTTP
110	TCP	POP3
161	UDP	SNMP
443	TCP	SSL
16,384-32,767	UDP	RTP-based Voice and Video

Sample TCP Port Numbers

20	FTP Data Channel
21	FTP Control Channel
23	Telnet
80	HTTP on WWW
135	RPC
139	NetBIOS Session Services

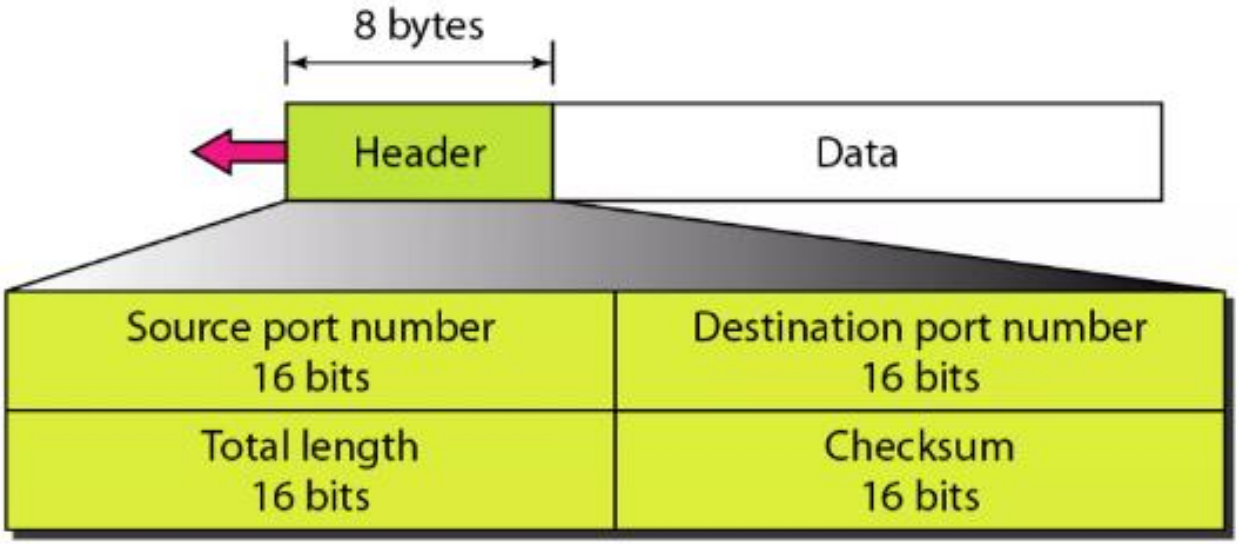
Note: There are port numbers applicable to UDP as well.

Transport Layer: USER DATAGRAM PROTOCOL (UDP)

User Datagram

- UDP packets, called user datagrams, have a fixed-size header of 8 bytes. Figure 23.9 shows the format of a user datagram.

Figure 23.9 *User datagram format*



Transport Layer: USER DATAGRAM PROTOCOL (UDP)

The fields are as follows:

- **Source port number**.
  - This is the port number used by the process running on the source host. It is **16 bits long**, which means that the port number can range from **0 to 65,535**.
  - If the source host is the **client** (a client sending a request), the port number, in most cases, is an **ephemeral port number** requested by the process and **chosen by the UDP software running on the source host.**
  - If the source host is the **server** (a server sending a response), the port number, in most cases, is a **well-known port number**.

Transport Layer: USER DATAGRAM PROTOCOL (UDP)

Destination port number.

- This is the port number used by the process running on the destination host. It is also **16 bits long**.
- If the destination host is the **server** (a client sending a request), the port number, in most cases, is a **well-known port number**.
- If the destination host is the **client** (a server sending a response), the port number, in most cases, is an ephemeral port number. In this case, the server copies the **ephemeral port number** it has received in the request packet.

Length.

This is a 16-bit field that defines the total length of the user datagram, header plus data. The 16 bits can define a total length of 0 to 65,535 bytes.

Checksum.

This field is used to detect errors over the entire user datagram (header plus data).

Transport Layer: UDP OPERATIONS

Connectionless Services

- Each user datagram sent by UDP is an **independent** datagram. There is **no relationship between** the different user datagrams even if they are coming from the same source process and going to the same destination program.
- The user datagrams are **not numbered**.
- There is **no connection establishment** and no connection termination, as is the case for TCP. This means that each user datagram can travel on a different path.

Flow and Error Control

- UDP is a very simple, unreliable transport protocol. There is **no flow control**. The receiver may overflow with incoming messages.
- There is **no error control** mechanism in UDP except for the checksum. This means that the sender does not know if a message has been lost or duplicated.
- When the receiver detects an error through the checksum, the user datagram is silently **discarded**.

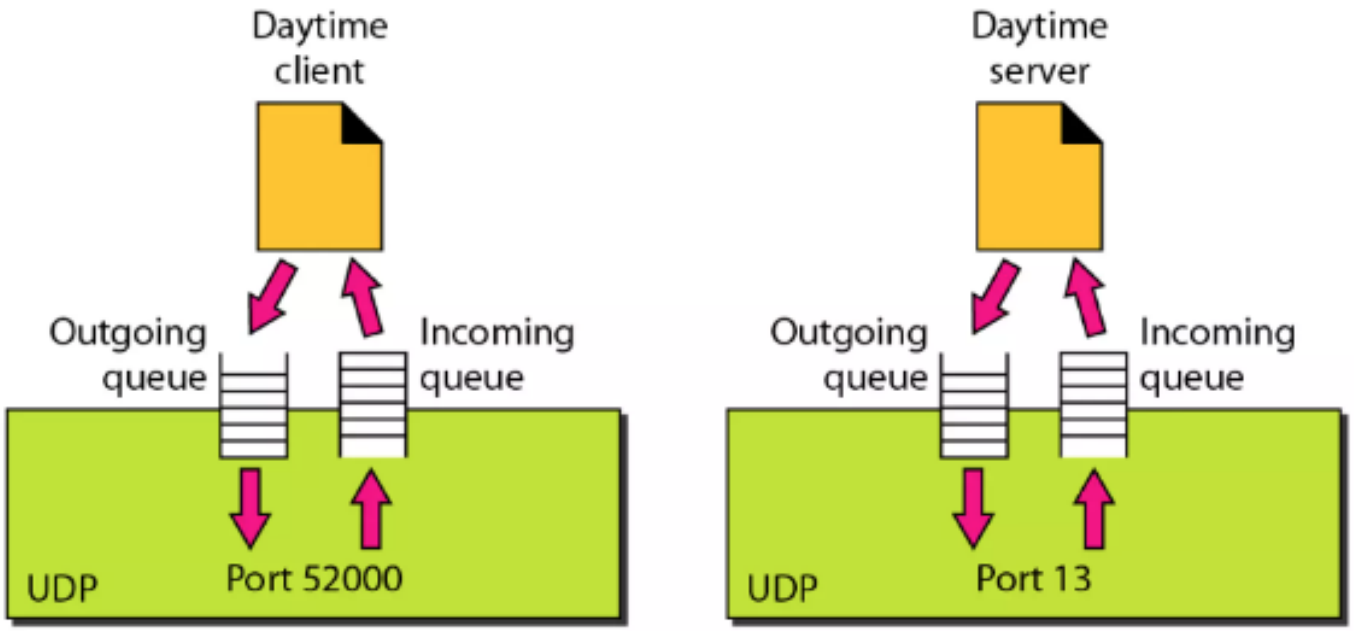
Transport Layer: UDP OPERATIONS

Queuing

- At the client site, when a process starts, it requests a port number from the operating system. Some implementations create both an incoming and an outgoing queue associated with each process. Other implementations create only an incoming queue associated with each process.
- The queues opened by the client are, in most cases, identified by ephemeral port numbers.
- The queues function as long as the process is running. When the process terminates, the queues are destroyed.

Transport Layer: USER DATAGRAM PROTOCOL (UDP)

Figure 23.12 *Queues in UDP*



Transport Layer: USER DATAGRAM PROTOCOL (UDP)

Typical Applications

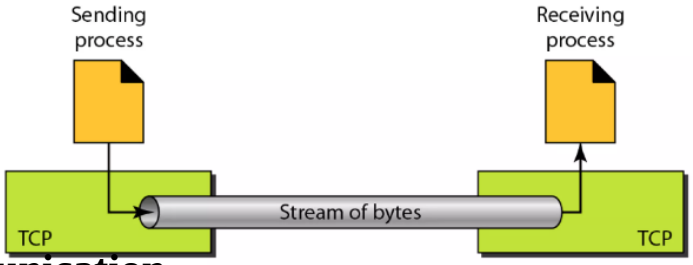
- The following shows some typical applications that can benefit more from the services of UDP than from those of TCP.
- UDP is suitable for a process that requires **simple request-response** communication with little concern for flow and error control.
- UDP is a suitable transport protocol for **multicasting**. Multicasting capability is embedded in the UDP software but not in the TCP software.
- UDP is used for management processes such as **SNMP**
- UDP is used for some route updating protocols such as **Routing Information Protocol (RIP)**
- UDP is normally used for interactive **real-time applications that cannot tolerate uneven** delay between sections of a received message

Transport Layer: Transmission Control Protocol (TCP)

TCP is called a connection-oriented, reliable transport protocol.

TCP Services

- **Process-to-Process Communication** :Like UDP, TCP provides process-to-process communication using port numbers.
- **Stream Delivery Service:** TCP creates an environment in which the two processes seem to be connected by an imaginary "tube“ that carries their data across the Internet.



Full-Duplex Communication

TCP offers full-duplex service, in which data can flow in both directions at the same time. Each TCP then has a **sending and receiving buffer**, and segments move in both directions.

Connection-Oriented Service

A process at site A wants to send and receive data from another process at site B, the following occurs:

1. The two TCPs establish a connection between them.
2. Data are exchanged in both directions.
3. The connection is terminated.

Reliable Service: TCP is a reliable transport protocol. It uses an **acknowledgment** mechanism to check the safe and sound arrival of data

Transport Layer: Transmission Control Protocol (TCP)

Numbering System: TCP software keeps track of the segments being transmitted or received, using two fields called the **sequence number and the acknowledgment number**.

Sequence Number: TCP assigns a sequence number to each segment that is being sent. The sequence number for each segment is the number of the **first byte carried** in that segment.

Example 23.3

Suppose a TCP connection is transferring a file of 5000 bytes. The first byte is numbered 1O,00l. What are the sequence numbers for each segment if data are sent in five segments, each carrying 1000 bytes?

Solution

The following shows the sequence number for each segment:

Segment 1	Sequence Number: 10,001 (range: 10,001 to 11,(00)
Segment 2	Sequence Number: 11,001 (range: 11,001 to 12,000)
Segment 3	Sequence Number: 12,001 (range: 12,001 to 13,000)
Segment 4	Sequence Number: 13,001 (range: 13,001 to 14,000)
Segment 5	Sequence Number: 14,001 (range: 14,001 to 15,000)

Transport Layer: Transmission Control Protocol (TCP)

Acknowledgement Number: The value of the acknowledgment field in a segment defines the **number of the next byte** a party expects to receive. The acknowledgment number is cumulative.

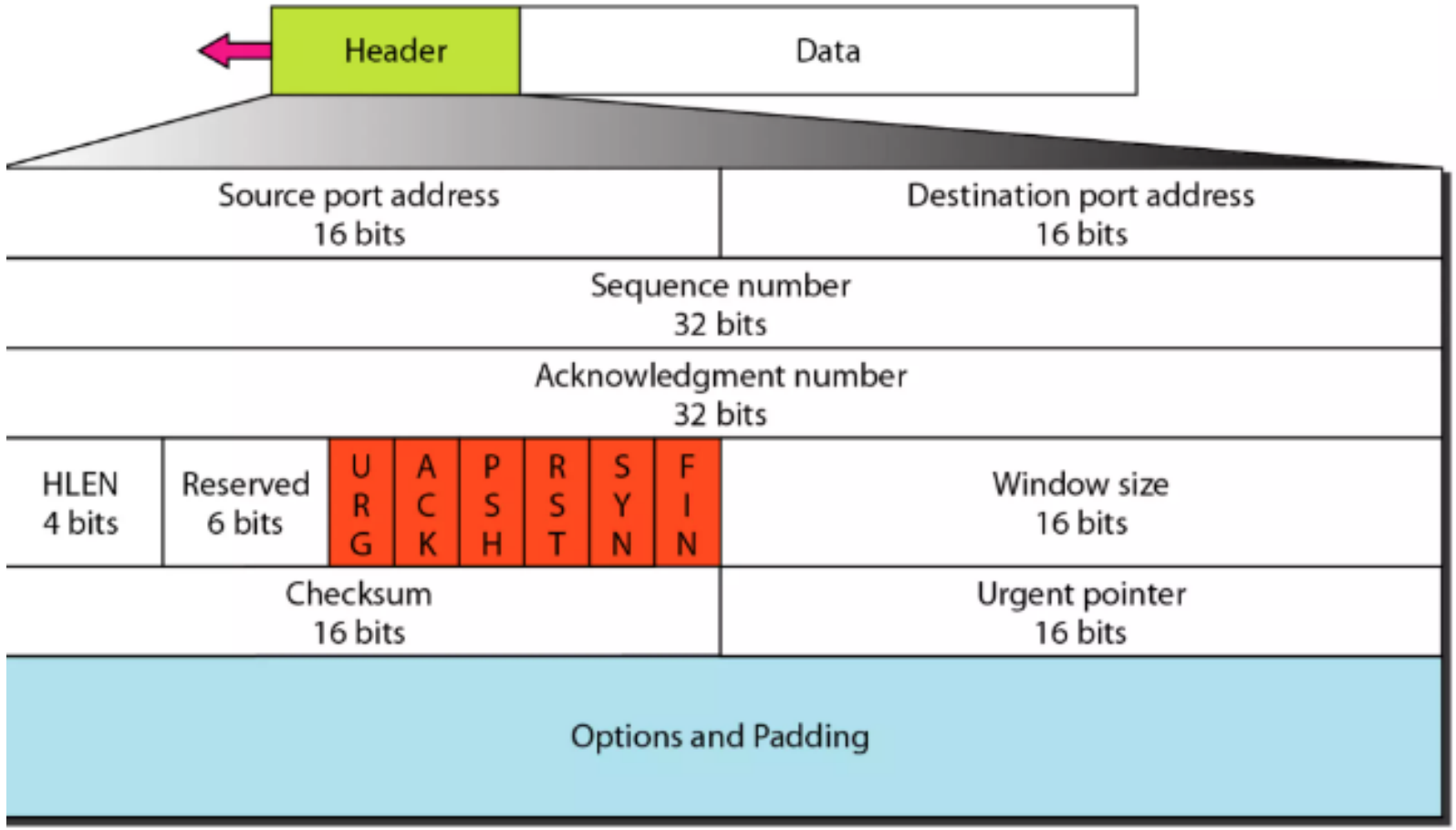
Flow Control : The receiver of the data controls the amount of data that are to be sent by the sender. This is done to prevent the receiver from being overwhelmed with data. The numbering system allows TCP to use a **byte-oriented flow control**.

Error Control: To provide reliable service, TCP implements an error control mechanism.

Congestion Control : TCP provides congestion control in the network. The amount of data sent by a sender is not only controlled by the receiver (flow control), but is also determined by the level of congestion in the network.

TCP Segment Header Format

Figure 23.16 *TCP segment format*



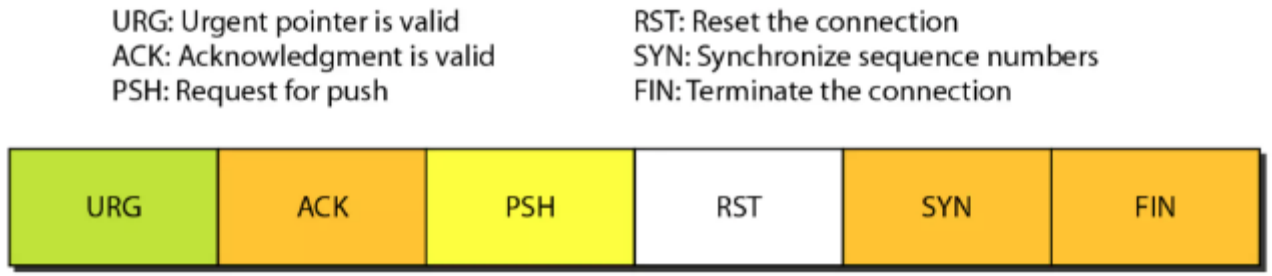
Transport Layer: Transmission Control Protocol (TCP)

The segment consists of a **20- to 60-byte header**, followed by data from the application program. The header is 20 bytes if there are no options and up to 60 bytes if it contains options.

Source port address. This is a 16-bit field that defines the port number of the application program in the host that is sending the segment.
Destination port address. This is a 16-bit field that defines the port number of the application program in the host that is receiving the segment.
Sequence number. This 32-bit field defines the number assigned to the first byte of data contained in this segment.
Acknowledgment number. This 32-bit field defines the byte number that the receiver of the segment is expecting to receive from the other party.
Header length. This 4-bit field indicates the number of 4-byte words in the TCP header. The length of the header can be between 20 and 60 bytes.
Reserved. This is a 6-bit field reserved for future use.
Control. This field defines 6 different control bits or flags as shown in Figure 23.17. One or more of these bits can be set at a time.

Transport Layer: Transmission Control Protocol (TCP)

Figure 23.17 *Control field*



Window size. This field defines the size of the window, in bytes, that the other party must maintain. The length of this field **is 16 bits**, so the **maximum size of the window is 65,535** bytes. This value is normally referred to as the **receiving window (rwnd)** and is determined by the receiver. The sender must obey the dictation of the receiver.

Checksum. This 16-bit field contains the checksum. The inclusion of the checksum in the TCP is mandatory.

Urgent pointer. This l6-bit field, which is valid only if the urgent flag is set, is used when the segment contains urgent data.

Options. There can be up to 40 bytes of optional information in the TCP header.

Transport Layer: Transmission Control Protocol (TCP)

A TCP Connection

- TCP establishes a **virtual path between** the source and destination.
- All the segments belonging to a message are then sent over this virtual path. Using a **single virtual pathway** for the entire message facilitates the **acknowledgment process as well as retransmission** of damaged or lost frames.
- **TCP, connection-oriented transmission requires three phases:**
  1. Connection establishment,
  2. Data transfer, and
  3. Connection termination.

Transport Layer: Transmission Control Protocol (TCP)

Connection Establishment

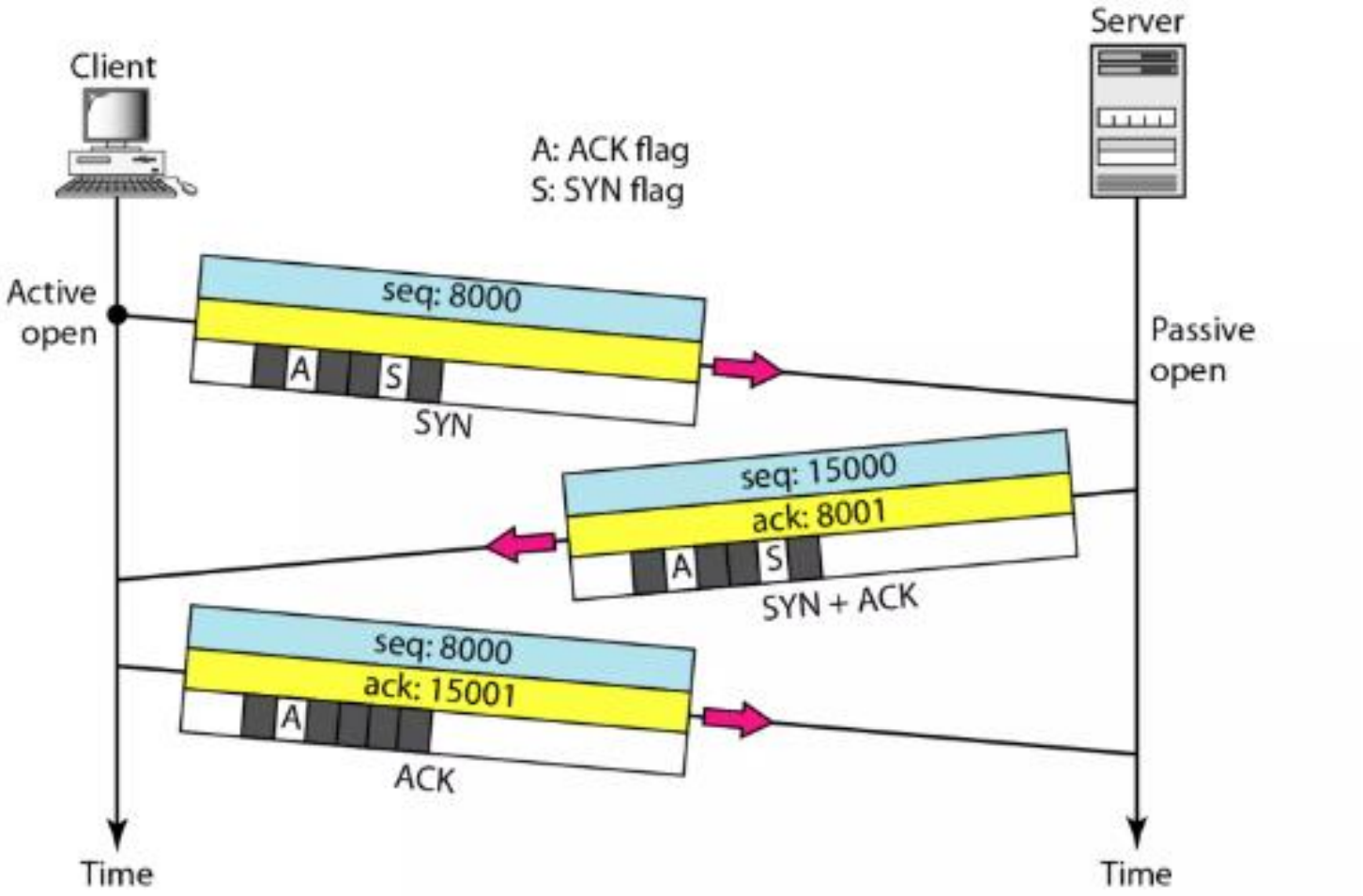
- TCP transmits data in **full-duplex mode.** When two TCPs in two machines are connected, they are able to send segments to each other simultaneously using **Three-Way Handshaking process: Connection Establishment, Data Transfer, Connection Termination.**
- The process starts with the server. The server program tells its TCP that it is ready to accept a connection. This is called a request for a **passive open**.
- The client program issues a request for an **active open.**

1. Connection Establishment:

- The client sends the first segment, a SYN segment, in which only the SYN flag is set.
- This segment is for synchronization of sequence numbers. It consumes one sequence number.
- The server sends the second segment, a SYN +ACK segment, with 2 flag bits set: SYN and ACK. This segment has a dual purpose. It is a SYN segment for communication in the other direction and serves as the acknowledgment for the SYN segment. It consumes one sequence number.
- The client sends the third segment. This is just an ACK segment. It acknowledges the receipt of the second segment with the ACK flag and acknowledgment number field.
- Sequence number in this segment is the same as the one in the SYN segment; the ACK segment does not consume any sequence numbers.

Transport Layer: Transmission Control Protocol (TCP)

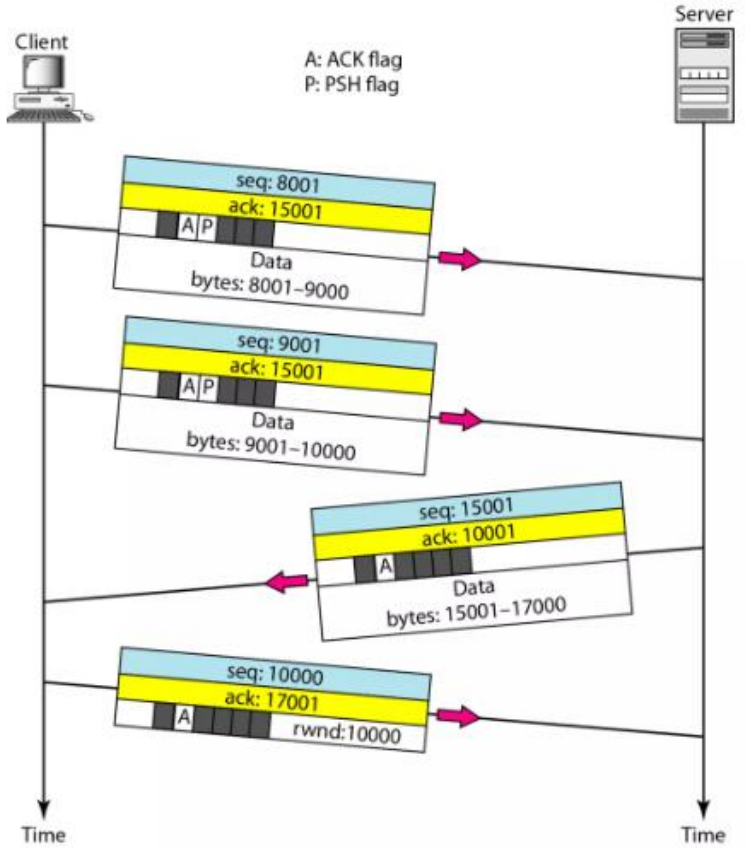
Figure 23.18 *Connection establishment using three-way handshaking*



II. Data Transfer

After connection is established, bidirectional data transfer can take place. The client and server can both send data and acknowledgments. The data segments sent by the client have the PSH (push) flag set so that the server TCP knows to deliver data to the server process as soon as they are received.

Figure 23.19 *Data transfer*

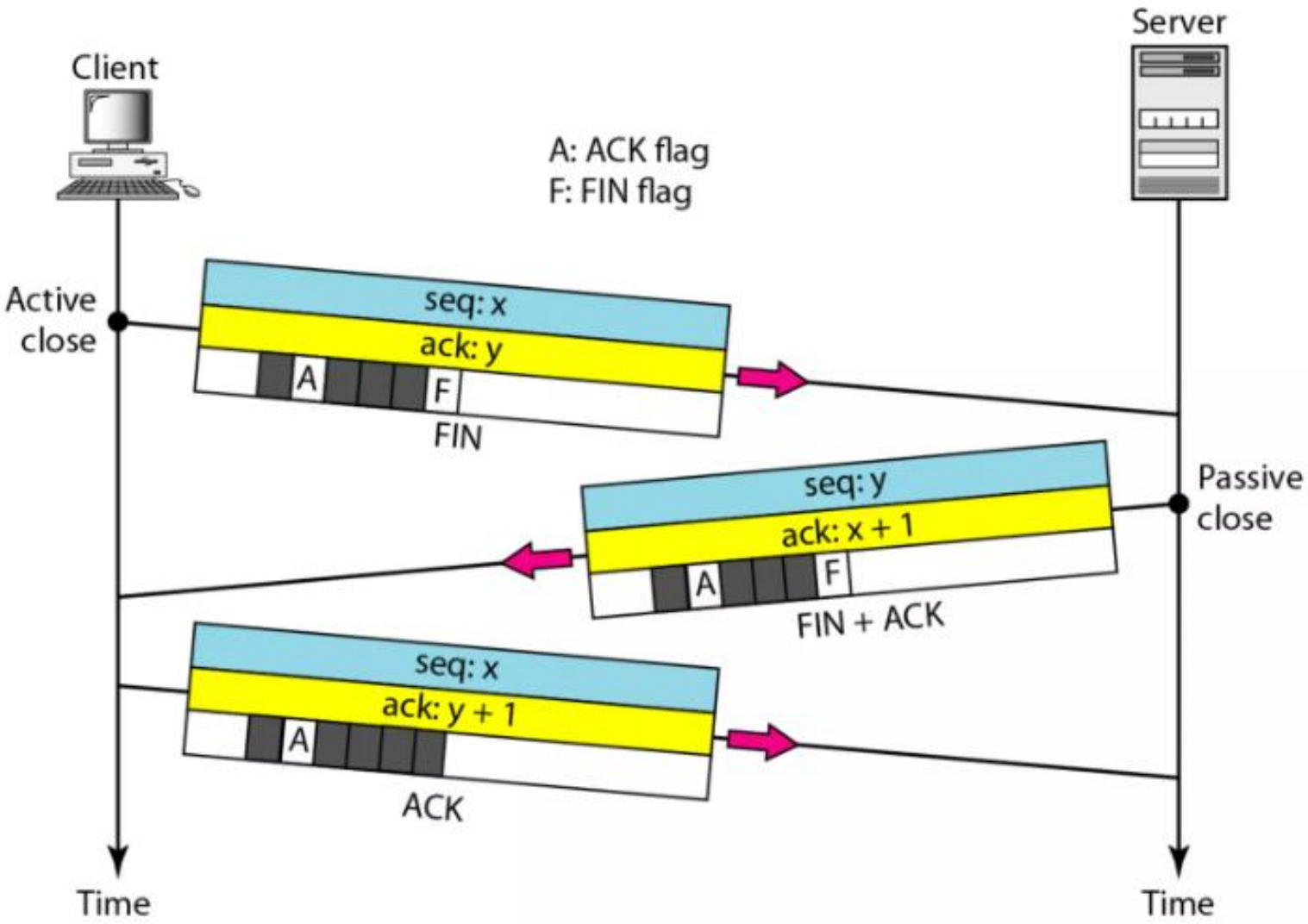


Connection Termination

Any of the two parties involved in exchanging data (client or server) can close the connection, initiated by the client. Two options for connection termination: **three-way handshaking and four-way handshaking with a half-close option.**

1. The client TCP, after receiving a close command from the client process, sends the first segment, a **FIN segment** in which the FIN flag is set.
2. The server TCP, after receiving the FIN segment, informs its process of the situation and sends the second segment, a **FIN +ACK** segment, to confirm the receipt of the FIN segment from the client and at the same time to announce the closing of the connection in the other direction.
3. The client TCP sends the last segment, an **ACK segment**, to confirm the receipt of the FIN segment from the TCP server.

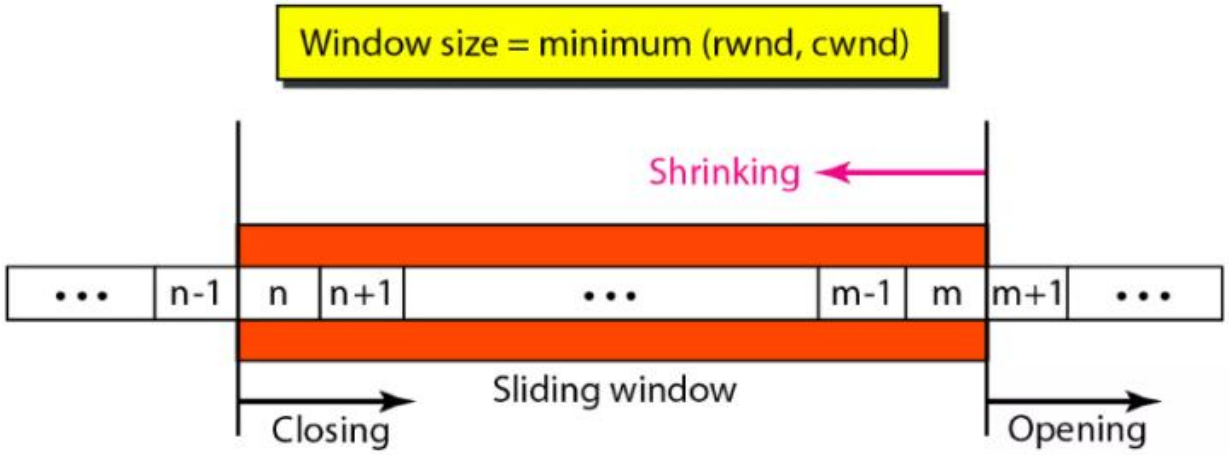
Figure 23.20 *Connection termination using three-way handshaking*



Flow Control

A sliding window is used to make transmission more efficient as well as to control the flow of data so that the destination does not become overwhelmed with data.
TCP sliding windows are byte-oriented.

Figure 23.22 *Sliding window*



- The window spans a portion of the buffer containing bytes received from the process. The bytes inside the window are the bytes that can be in transit.
- The imaginary window has two walls: one left and one right.

- The window is opened, closed, or shrunk.
- These three activities, as we will see, are in the control of the receiver (and depend on congestion in the network), not the sender.
- The sender must obey the commands of the receiver in this matter. Opening a window means moving the right wall to the right. This allows more new bytes in the buffer that are eligible for sending.
- Closing the window means moving the left wall to the right. This means that some bytes have been acknowledged and the sender

Figure 23.23 *Example 23.6*